

UNIVERSIDADE REGIONAL DE BLUMENAU
CENTRO DE CIÊNCIAS EXATAS E NATURAIS
CURSO DE SISTEMAS DE INFORMAÇÃO – BACHARELADO

OTIMIZAÇÃO DE PORTFÓLIOS PERSONALIZADOS COM
ALGORITMO GENÉTICO MULTIOBJETIVO

ALEXANDRE SILVA ZABEL
BRUNO FERRARI VINCENSI

BLUMENAU
2025

ALEXANDRE SILVA ZABEL E BRUNO FERRARI VICENSI

OTIMIZAÇÃO DE PORTFÓLIOS PERSONALIZADOS COM ALGORITMO GENÉTICO MULTIOBJETIVO

Trabalho de Conclusão de Curso apresentado ao curso de graduação em Sistemas de Informação do Centro de Ciências Exatas e Naturais da Universidade Regional de Blumenau como requisito parcial para a obtenção do grau de Bacharel em Sistemas de Informação.

Prof. Aurélio Faustino Hoppe, Mestre - Orientador

Esta página deverá ser substituída pela folha de assinaturas entregue na Banca.

Digitalize a folha e cole aqui para a entrega da versão final do TCC.

Atenção: não ultrapasse as margens!

Dedicamos este trabalho à todas as pessoas que de alguma forma contribuíram positivamente para o nosso processo de graduação.

AGRADECIMENTOS

Com gratidão, dedicamos este trabalho:

Às nossas famílias, que sempre nos ofereceram apoio incondicional em cada escolha e decisão tomada.

Às nossas namoradas, pelo carinho, paciência e compreensão em todos os momentos, especialmente nos mais desafiadores.

Aos nossos amigos, que de alguma forma contribuíram para o nosso crescimento ao longo dos últimos anos.

Ao nosso orientador, professor Aurélio Faustino Hoppe, pelo suporte e valiosas orientações que foram essenciais para a realização deste trabalho.

Todo progresso ocorre fora da zona de conforto.

Michael John Bobak

RESUMO

A otimização de portfólios é um desafio recorrente para investidores que buscam equilibrar retorno e risco diante de restrições práticas e diferentes perfis de investimento. Decisões manuais ou heurísticas tendem a ser limitadas frente à necessidade de personalização e ao controle simultâneo de múltiplos objetivos. Para enfrentar esse problema, este trabalho apresenta o desenvolvimento de uma plataforma de otimização baseada em um Algoritmo Genético Multiobjetivo (AGMO), implementado com o suporte da biblioteca pymoo, utilizando a especialização R-NSGA-II para direcionar a busca conforme o perfil do investidor. O processo inicia-se com a extração automatizada de séries históricas via API da Yahoo Finance, realizando a limpeza, alinhamento e cálculo de retornos, que subsidiam o cálculo das estatísticas centrais do problema, como retornos médios e matriz de covariância. A otimização considera três objetivos — retorno, variância e CVaR — resolvidos sobre populações iniciais geradas por amostragem no Simplex com distribuição de Dirichlet, enquanto operadores especializados asseguram soma unitária, não-negatividade e respeito à cardinalidade. A seleção guiada por referência permite identificar automaticamente carteiras alinhadas às preferências do usuário. A análise dos hiperparâmetros, incluindo população e número de gerações, foi conduzido com suporte do indicador R-Hypervolume para garantir fronteiras mais amplas e bem distribuídas. Nos testes, observou-se que as carteiras produzidas atenderam de forma consistente os diferentes perfis de risco, além de apresentarem desempenho competitivo quando comparadas a *benchmarks* de mercado em *backtests*. A solução foi disponibilizada através de uma API REST, construída utilizando o *framework* Flask, integrada a uma aplicação web em React, nomeada como WalletAI, permitindo autenticação, ajuste de parâmetros e criação de carteiras personalizadas. A avaliação do sistema foi realizada por meio de testes de usabilidade, que indicaram uma boa aceitação da interface e das funcionalidades, destacando a clareza e a facilidade de uso da plataforma. Conclui-se que a plataforma oferece uma solução reprodutível e personalizável para geração de carteiras alinhadas a perfis de risco distintos, integrando desde a extração automatizada dos dados até a recomendação final, e aponta direções claras para extensões e aplicação prática.

Palavras-chave: Investimentos financeiros. Diversificação. Carteira. Perfil de risco. Otimização de portfólios. AGMO. R-NSGA-II. Flask. React.

ABSTRACT

Portfolio optimization is a recurring challenge for investors seeking to balance return and risk while accounting for practical constraints and different investment profiles. Manual or heuristic decision making tends to be limited when faced with the need for personalization and the simultaneous control of multiple objectives. To address this problem, this work presents the development of an optimization platform based on a Multi-Objective Genetic Algorithm (MOGA), implemented with support from the pymoo library and using the R-NSGA-II specialization to guide the search according to the investor's profile. The process begins with automated extraction of historical time series via the Yahoo Finance API, followed by cleaning, alignment, and return computation, which support the calculation of central statistical components such as mean returns and the covariance matrix. The optimization considers three objectives — return, variance, and CVaR — solved over initial populations generated through Simplex sampling with a Dirichlet distribution, while specialized operators ensure unit-sum weights, non-negativity, and compliance with cardinality constraints. Reference-guided selection enables the automatic identification of portfolios aligned with the user's preferences. Hyperparameter analysis, including population size and number of generations, was conducted using the R-Hypervolume indicator to ensure broader and well-distributed Pareto fronts. In the experiments, the resulting portfolios consistently met different risk profiles and demonstrated competitive performance when compared to market benchmarks in backtests. The solution was made available through a REST API built with the Flask framework and integrated into a React-based web application named WalletAI, enabling authentication, parameter adjustment, and personalized portfolio creation. System evaluation was conducted through usability tests, which showed strong acceptance of the interface and features, highlighting the platform's clarity and ease of use. It is concluded that the platform provides a reproducible and customizable solution for generating portfolios aligned with distinct risk profiles, integrating all stages from automated data acquisition to final recommendation, while also pointing to clear directions for extensions and practical application.

Keywords: Financial investments. Diversification. Portfolio. Risk profile. Portfolio optimization. MOGA. R-NSGA-II. Flask. React.

LISTA DE FIGURAS

Figura 1 – Representação gráfica do conjunto de combinações.....	18
Figura 2 – Diagrama de fluxo do NSGA-II.....	20
Figura 3 – Exemplo das soluções não dominadas do NSGA-II e R-NSGA-II	22
Figura 4 – Demonstração de como o ponto de referência altera o comportamento das soluções na Fronteira de Pareto do R-NSGA-II	22
Figura 5 – MER do banco de dados.....	28
Figura 6 – Diagrama de atividades para a otimização de portfólio	30
Figura 7 – Arquivo .csv de ativos.....	30
Figura 8 – <i>Pipeline</i> de desenvolvimento do AGMO	33
Figura 9 – Comportamento do CVaR em uma carteira	37
Figura 10 – Comparativo de duas configurações	44
Figura 11 – Fronteiras de Pareto com base no perfil	45
Figura 12 – Soluções normalizadas com o <i>reference point</i>	47
Figura 13 – Comparações entre a carteira gerada e o <i>benchmark</i>	48
Figura 14 – Retornos individuais dos ativos das carteiras	49
Figura 15 – Demonstração do fluxo de autenticação e uso do modelo AGMO	50
Figura 16 – Diagrama de casos de uso.....	52
Figura 17 – Tela de login	55
Figura 18 – Tela de cadastro de usuário.....	56
Figura 19 – <i>Dashboard</i> para visualização das carteiras e métricas	57
Figura 20 – Tela de exibição das carteiras	58
Figura 21 – Tela de criação de carteiras.....	59
Figura 22 – Tela exibição do usuário	60
Figura 23 – <i>Pop-up</i> de confirmação de exclusão de conta.....	61

LISTA DE QUADROS

Quadro 1 – Pseudocódigo código do R-NSGA-II	23
Quadro 2 – Resultado de buscas por termos contidos em artigos em língua inglesa	24
Quadro 3 – Síntese dos trabalhos correlatos selecionados	25
Quadro 4 – Trecho do código para persistência dos ativos	31
Quadro 5 – Trecho do código para a extração dos dados do <i>yfinance</i>	31
Quadro 6 – <i>DataFrame</i> de um período de 6 meses da ação BBAS3.....	32
Quadro 7 – Trecho de código da execução do agendador	32
Quadro 8 – Trecho de código da preparação dos dados.....	33
Quadro 9 – Exemplo das linhas retornadas do banco de dados	34
Quadro 10 – Exemplo da transformação de dados para a matriz	34
Quadro 11 – Filtro com base no horizonte de investimento	34
Quadro 12 – Cálculo das médias de retorno e matriz de covariância.	35
Quadro 13 – Matriz de covariância de 5 ações.....	35
Quadro 14 – Cálculos das funções de <i>Fitness</i>	36
Quadro 15 – Cálculo do CVaR.....	36
Quadro 16 – Algoritmo R-NSGA-II.....	38
Quadro 17 – Implementação da função <i>_enforce_cardinality</i>	39
Quadro 18 – Implementação da limitação de pesos.....	39
Quadro 19 – Implementação da geração inicial	40
Quadro 20 – Exemplo da geração de uma população inicial	40
Quadro 21 – Implementação do <i>crossover</i> usando SBX	41
Quadro 22 – Exemplo de um <i>crossover</i> de dois pais.....	41
Quadro 23 – Transferência de peso entre ativos.....	42
Quadro 24 – Mutação por substituição: troca um ativo por um inativo	42
Quadro 25 – Cálculo do indicador R2	43
Quadro 26 – Escolha da melhor carteira das soluções.....	46
Quadro 27 – Requisição de login da API.....	50
Quadro 28 – Exemplo do <i>handler</i> de login em <i>Login.jsx</i>	56
Quadro 29 – Exemplo de submissão em <i>Register.jsx</i>	57
Quadro 30 – Busca de resumo no <i>Dashboard.jsx</i>	58
Quadro 31 – Listagem e seleção em <i>Carteiras.jsx</i>	59

Quadro 32 – Carregar detalhe utilizando <code>PieChart</code> em <code>CarteiraDetail.jsx</code>	60
Quadro 33 – Salvar alterações e deletar conta em <code>Profile.jsx</code>	60
Quadro 34 – Função de <i>logout</i> em <code>UserPopover.jsx</code> ou similar.....	61

LISTA DE TABELAS

Tabela 1 – Perfil dos usuários	63
Tabela 2 – Respostas da execução da lista de tarefas	63
Tabela 3 – Respostas referentes as funcionalidades da aplicação	64

LISTA DE ABREVIATURAS E SIGLAS

API – Application Programming Interface

ASF – Achievement Scalarizing Function

CSS – Cascading Style Sheets

CVaR – Conditional Value at Risk

ES Modules – ECMAScript Modules

HTTP – Hypertext Transfer Protocol

JSON – JavaScript Object Notation

JWT – JSON Web Token

NSGA-II – Non-dominated Sorting Genetic Algorithm II

REST – REpresentational State Transfer

RF – Requisito Funcional

RNF – Requisito Não Funcional

R-NSGA-II – Reference Point Based Non-dominated Sorting Genetic Algorithm II

SBX – Simulated Binary Crossover

SGBD – Database Management System

SPA – Single Page Application

UC – Use Case

UI – User Interface

SUMÁRIO

1 INTRODUÇÃO	14
1.1 OBJETIVOS.....	16
1.2 ESTRUTURA.....	16
2 FUNDAMENTAÇÃO TEÓRICA.....	17
2.1 DIVERSIFICAÇÃO E TEORIA MODERNA DO PORTFÓLIO	17
2.2 ALGORITMOS GENÉTICOS MULTIOBJETIVO (AGMOS).....	19
2.3 TRABALHOS CORRELATOS	24
3 DESENVOLVIMENTO.....	27
3.1 MODELO DE OTIMIZAÇÃO DE PORTFÓLIOS (BACKEND)	27
3.1.1 Requisitos.....	27
3.1.2 Modelagem do banco de dados	28
3.1.3 Diagrama de atividades	29
3.1.4 Extração de dados	30
3.1.5 Implementação do AGMO	32
3.1.6 Definição dos hiperparâmetros.....	42
3.1.7 API de acesso ao modelo AGMO.....	49
3.2 PLATAFORMA WEB (FRONTEND).....	51
3.2.1 Diagrama de Casos de Uso.....	52
3.2.2 Requisitos Funcionais e Não Funcionais	53
3.2.3 Operacionalidade da aplicação web.....	55
3.2.4 Teste de Usabilidade	61
4 CONCLUSÕES	66
REFERÊNCIAS.....	69

1 INTRODUÇÃO

A inteligência artificial (IA) tem transformado significativamente o mercado financeiro, permitindo a automação de processos, a realização de análises mais sofisticadas e a geração de informações precisas para investidores (Garcez, 2022). Ferramentas baseadas em IA conseguem prever tendências de mercado, gerenciar carteiras de investimento e avaliar riscos, promovendo maior eficiência e confiabilidade nas decisões financeiras (Públio, 2022).

Com o aumento da complexidade dos mercados e o volume crescente de informações, identificar as melhores oportunidades de investimento tornou-se um grande desafio. Métodos tradicionais frequentemente não conseguem lidar com a alta dimensionalidade dos dados e a volatilidade do mercado (Garcez, 2022). Nesse cenário, a IA se destaca por sua capacidade de processar grandes volumes de dados financeiros em tempo real, identificando padrões que podem passar despercebidos em análises manuais. Além disso, ela reduz o viés humano, fornecendo recomendações baseadas em dados concretos e objetivos (Investing.com Academy, 2024).

Neste contexto, sistemas automatizados de recomendação de investimentos têm se mostrado soluções promissoras, personalizando alocações de acordo com o perfil de cada investidor. Esses sistemas utilizam IA para analisar variáveis como preferências individuais, tolerância ao risco e objetivos financeiros, oferecendo estratégias otimizadas (Pavão, 2023). Além disso, incorporam técnicas matemáticas, como programação linear e métodos de otimização, para avaliar múltiplos critérios, como risco ajustado ao retorno e diversificação da carteira (Cornuéjols; Tutuncu, 2007, Vieira, 2021).

Diante dessa combinação de fatores e da necessidade de equilibrar objetivos conflitantes, a escolha do método de otimização torna-se particularmente relevante para o sucesso das recomendações de investimentos. Entre as opções mais reconhecidas, destaca-se o Non-dominated Sorting Genetic Algorithm II (NSGA-II), amplamente utilizado devido à sua eficiência na exploração da fronteira de Pareto e na preservação da diversidade das soluções. Outras alternativas igualmente relevantes incluem o Strength Pareto Evolutionary Algorithm 2 (SPEA2), conhecido por sua robustez em problemas com múltiplos objetivos, e o Multiobjective Evolutionary Algorithm based on Decomposition (MOEA/D), que se destaca por seu enfoque em métodos de decomposição.

Além disso, existe a possibilidade de aprimorar os resultados combinando algoritmos genéticos com outras técnicas avançadas, como aprendizado de máquina e heurísticas

específicas. Essa integração pode aumentar a precisão, melhorar a eficiência das soluções geradas e criar um sistema mais adaptável às necessidades diversificadas dos usuários.

Entretanto, apesar desses avanços, ainda existem desafios significativos relacionados à acessibilidade e à necessidade de conhecimento técnico para interpretar os dados gerados. Muitos investidores, especialmente os menos experientes, enfrentam dificuldades para compreender conceitos financeiros complexos, como fronteira eficiente, risco ajustado ao retorno e diversificação, que frequentemente aparecem nos relatórios e análises gerados por essas ferramentas. A apresentação inadequada dos dados, por sua vez, pode dificultar o engajamento do usuário e limitar o potencial de adoção dessas tecnologias.

Além disso, o viés técnico de muitas soluções disponíveis no mercado pressupõe um nível avançado de familiaridade com jargões financeiros e matemáticos, criando barreiras para investidores iniciantes ou para aqueles que preferem uma abordagem mais prática. Portanto, é essencial que as interfaces sejam desenvolvidas de forma intuitiva e didática, permitindo que usuários de diferentes níveis de experiência compreendam facilmente as recomendações e tomem decisões informadas.

Neste sentido, o progresso contínuo nas interfaces tem avançado significativamente para superar esses desafios. Tecnologias modernas, como *dashboards* interativos, visualizações dinâmicas e explicações em linguagem natural, estão tornando essas ferramentas mais amigáveis e acessíveis para um público mais amplo. Recursos como personalização da interface, tutoriais integrados e exemplos práticos ajudam a reduzir a complexidade e melhoram a experiência do usuário. Essas inovações têm o potencial de democratizar o uso de estratégias avançadas de investimento, permitindo que mesmo investidores com pouca experiência ou conhecimento técnico se beneficiem de soluções sofisticadas e otimizadas (Públio, 2022).

Nesse panorama, percebe-se que, embora os avanços em IA e otimização tenham ampliado significativamente as possibilidades para recomendações automatizadas de investimento, a eficácia dessas soluções ainda depende de sua capacidade de apresentar informações de forma clara, acessível e alinhada ao perfil do usuário. Assim, torna-se fundamental desenvolver abordagens que conciliem rigor técnico, personalização e facilidade de compreensão. A partir disso, este trabalho tem como objetivo geral apresentar uma solução que integra técnicas avançadas de otimização com uma interface intuitiva, capaz de apoiar investidores de diferentes níveis de experiência na tomada de decisões mais informadas e personalizadas.

1.1 OBJETIVOS

O objetivo principal deste trabalho consiste em disponibilizar uma plataforma web para a otimização de portfólios de investimento utilizando Algoritmos Genéticos MultiObjetivo (AGMO), capaz de considerar simultaneamente múltiplos critérios de decisão (retorno esperado, risco, diversificação e restrições do investidor) e personalizar recomendações de acordo com o perfil individual do usuário.

Os objetivos específicos são:

- a) modelar o problema da alocação de ativos como um problema de otimização multiobjetivo, considerando funções de retorno, risco e diversificação;
- b) implementar e testar diferentes configurações do algoritmo genético multiobjetivo e variantes dinâmicas, analisando impacto de parâmetros como tamanho da população, taxas de cruzamento e mutação, e número de gerações;
- c) validar o desempenho do aplicativo por meio de *backtesting* histórico e comparação com *benchmarks* relevantes como o Ibovespa, analisando métricas de retorno, risco e eficiência de diversificação;
- d) avaliar diferentes critérios de decisão e restrições de portfólio, incluindo limites por classe de ativo, ativos indesejados e horizonte de investimento;
- e) disponibilizar o modelo desenvolvido em uma plataforma web para avaliar a aderência do sistema a diferentes perfis de investidor.

1.2 ESTRUTURA

A organização deste trabalho está distribuída em quatro capítulos. O primeiro apresenta a introdução e os objetivos da pesquisa. O segundo reúne a fundamentação teórica, contemplando os principais conceitos e os estudos relacionados. O terceiro capítulo aborda o desenvolvimento da ferramenta, descrevendo os requisitos funcionais e não funcionais, as técnicas e bibliotecas empregadas, bem como a implementação do protótipo e os resultados obtidos. Por fim, o quarto capítulo expõe a conclusão, relacionando expectativas e achados, além de discutir limitações e propor direções para investigações futuras.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta uma visão geral dos tópicos que servem como base para o estudo a ser desenvolvido. A seção 2.1 aborda a diversificação de carteiras de investimentos e explora os fundamentos matemáticos e evolutivos da Teoria Moderna do Portfólio, enquanto a seção 2.2 aborda os Algoritmos Genéticos MultiObjetivo (AGMO). A seção 2.3 apresenta uma revisão dos trabalhos correlatos, que são considerados similares ao projeto proposto.

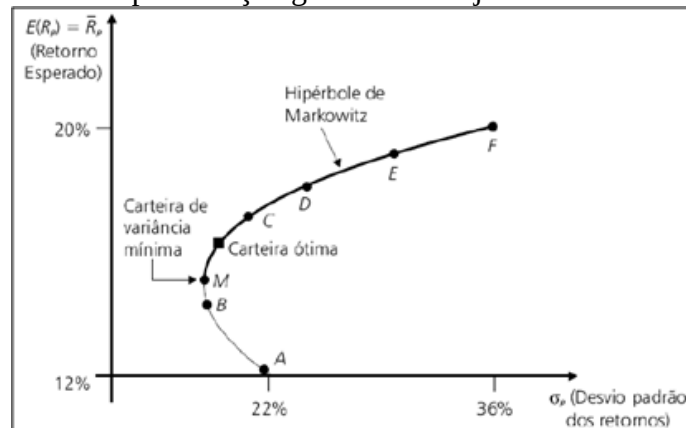
2.1 DIVERSIFICAÇÃO E TEORIA MODERNA DO PORTFÓLIO

Uma carteira de investimentos é definida como um conjunto estruturado de ativos financeiros selecionados com o objetivo de alcançar retornos consistentes, respeitando níveis de risco alinhados ao perfil, aos objetivos e ao horizonte temporal do investidor (Markowitz, 1952). A construção de carteiras envolve decisões estratégicas e táticas de alocação entre diferentes classes de ativos, tais como ações, títulos públicos e privados, fundos imobiliários, commodities e aplicações alternativas (Elton *et al.*, 2014). Estudos demonstram que carteiras diversificadas podem proporcionar ganhos reais médios de 4-6% ao ano no longo prazo, superando significativamente aplicações convencionais (Elton *et al.*, 2014).

A eficácia da diversificação pode ser analisada através da matriz de correlação, que quantifica as relações lineares entre os retornos dos diferentes ativos, onde valores próximos de -1 indicam movimentos opostos (diversificação ideal), enquanto correlações próximas a +1 sugerem comovimentos que limitam os benefícios da diversificação (Fabozzi; Focardi; Kolm, 2007). A análise dessas correlações permite a construção de carteiras com risco sistemático minimizado, sem comprometer o retorno esperado.

O marco teórico para a construção científica de carteiras de investimento foi estabelecido por Harry Markowitz em 1952, por meio da Teoria Moderna do Portfólio (TMP), que revolucionou a forma como se entende a relação entre risco e retorno (Markowitz, 1952). Sua principal contribuição foi demonstrar que o risco de uma carteira não é a simples soma dos riscos individuais dos ativos, mas depende da forma como esses ativos se correlacionam entre si, evidenciando o valor da diversificação para a redução do risco sem necessariamente comprometer o retorno (Elton *et al.*, 2014). A partir disso, surge o conceito da fronteira eficiente, ilustrada na Figura 1, que representa o conjunto de carteiras que oferecem o maior retorno possível para cada nível de risco assumido, ou, alternativamente, o menor risco possível para um dado nível de retorno esperado (Bodie; Kane; Marcus, 2014).

Figura 1 – Representação gráfica do conjunto de combinações



Fonte: Assaf Neto (2008).

Matematicamente, a teoria se baseia em dois pilares fundamentais. O primeiro é o cálculo do retorno esperado da carteira, expresso pela Equação 1.

$$E(R_p) = \sum w_i E(R_i) \quad (1)$$

onde w_i representa o peso de cada ativo e $E(R_i)$ o retorno esperado do ativo i (Markowitz, 1952). O segundo é o risco da carteira, representado pela Equação 2, que considera não apenas a variância individual dos ativos, mas também a covariância entre eles (Elton *et al.*, 2014).

$$\sigma_p^2 = \sum \sum w_i w_j \sigma_{ij} \quad (2)$$

Esse modelo evidenciou que quanto menor a correlação entre os ativos, maior o benefício da diversificação, pois os movimentos opostos entre eles tendem a suavizar a volatilidade total da carteira (Bodie; Kane; Marcus, 2014).

Apesar de sua relevância, a Teoria Moderna do Portfólio apresenta limitações quando aplicada ao mundo real. Ela assume que os retornos seguem distribuição normal, que os investidores são perfeitamente racionais e que os parâmetros estatísticos utilizados (retornos esperados, variâncias e covariâncias) são conhecidos com precisão, o que raramente ocorre na prática (Oliveira, 2010). Além disso, fatores como custos de transação, liquidez, tributação e mudanças nas condições econômicas não são contemplados no modelo clássico, o que pode comprometer sua aplicabilidade em cenários reais de mercado (Santos, 2010).

Essas limitações, segundo Oliveira (2010) se tornam ainda mais evidentes em um ambiente financeiro marcado por alta volatilidade, eventos extremos e comportamentos não lineares, características que desafiam diretamente as premissas de normalidade e estabilidade estatística adotadas pela teoria. Em situações de crise, por exemplo, os retornos apresentam caudas pesadas e correlações entre ativos tendem a aumentar abruptamente, reduzindo a eficácia da diversificação tradicional proposta por Markowitz. Ao mesmo tempo, estudos de finanças comportamentais mostram que os investidores frequentemente tomam decisões

influenciadas por emoções, vieses cognitivos e heurísticas, contrariando a hipótese de racionalidade plena assumida nos modelos clássicos.

Além disso, de acordo com Elton *et al.* (2014), a dinâmica contemporânea dos mercados, impulsionada pela velocidade das negociações eletrônicas e pela disponibilidade massiva de dados, exige modelos capazes de se adaptar rapidamente às mudanças. Segundo os autores, a Teoria Moderna do Portfólio, com sua estrutura estática e dependência de estimativas históricas, apresenta dificuldades em capturar esses padrões dinâmicos. A instabilidade dos parâmetros ao longo do tempo pode gerar carteiras subótimas ou excessivamente sensíveis a pequenas alterações nas estimativas, reduzindo a confiabilidade das recomendações geradas.

Nesse cenário, Bodie, Kane e Marcus (2014) destacam a necessidade de abordagens mais flexíveis, capazes de incorporar múltiplos objetivos, incertezas e restrições reais do mercado. Essas abordagens incluem métodos heurísticos, algoritmos evolutivos e técnicas de aprendizado de máquina, que se destacam por sua capacidade de lidar com problemas complexos, não lineares e de alta dimensionalidade. Assim, a busca por alternativas complementares ou substitutas à Teoria Moderna do Portfólio reflete uma demanda por modelos mais realistas, robustos e aderentes ao comportamento efetivo dos mercados financeiros.

2.2 ALGORITMOS GENÉTICOS MULTIOBJETIVO (AGMOS)

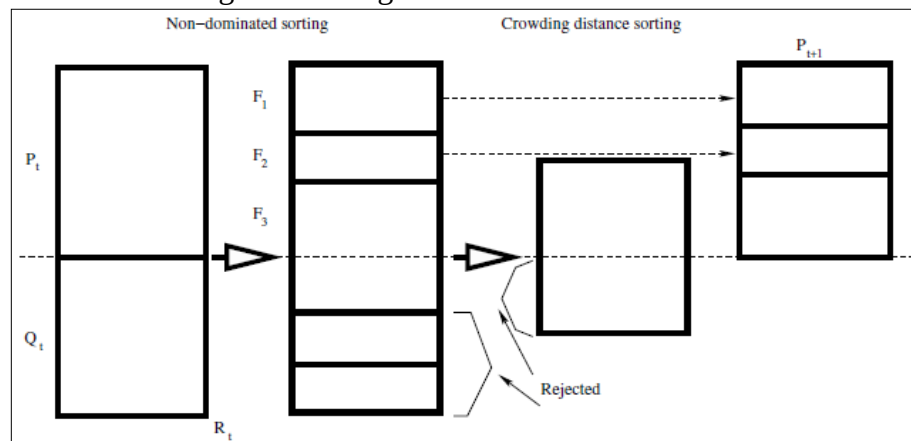
Os Algoritmos Genéticos MultiObjetivos (AGMOs) constituem uma abordagem evolucionária amplamente utilizada para a resolução de problemas complexos de otimização que envolvem múltiplos objetivos conflitantes (Deb *et al.*, 2002). Inspirados nos princípios da seleção natural propostos por Darwin (1859), esses algoritmos simulam o processo de evolução biológica por meio de populações de soluções que se transformam ao longo das gerações, buscando aprimorar a qualidade das soluções encontradas (Goldberg, 1989).

Diferentemente dos métodos tradicionais de otimização, que geralmente lidam com um único objetivo, os AGMOs são projetados para buscar soluções que equilibram múltiplos critérios simultaneamente (Zitzler; Thiele, 1999). O conceito de dominância de Pareto, originalmente formulado por Pareto (1906), é central nesse processo: uma solução é considerada não dominada quando não existe outra que seja melhor em todos os objetivos ao mesmo tempo (Deb *et al.*, 2002). Essa característica permite a construção de um conjunto de soluções eficientes, conhecido como fronteira de Pareto, no qual cada alternativa representa um compromisso viável entre os objetivos conflitantes (Coello; Lamont; Van Veldhuizen, 2007).

O funcionamento dos AGMOs baseia-se em três operadores principais: (i) seleção, (ii) cruzamento (*crossover*) e (iii) mutação (Goldberg, 1989). Inicialmente, uma população de soluções aleatórias é gerada, na qual cada indivíduo representa uma possível resposta ao problema (Deb *et al.*, 2002). Esses indivíduos são avaliados de acordo com os objetivos definidos, e os mais promissores são selecionados para reprodução por meio de mecanismos como torneio ou roleta (Michalewicz, 1996). O cruzamento gera novas soluções combinando características dos indivíduos selecionados, enquanto a mutação introduz variações aleatórias para manter a diversidade genética da população, evitando a convergência prematura (Goldberg, 1989). Com o passar das gerações, o algoritmo converge para um conjunto de soluções cada vez mais próximas do ótimo multiobjetivo (Deb *et al.*, 2002).

Dentre os AGMOs mais reconhecidos, destaca-se o Non-dominated Sorting Genetic Algorithm II (NSGA-II), desenvolvido por Deb *et al.* (2002), amplamente utilizado devido à sua eficiência na ordenação das soluções por níveis de dominância e à preservação da diversidade por meio da métrica conhecida como *crowding distance*. Essa métrica permite manter uma distribuição uniforme das soluções ao longo da fronteira de Pareto, evitando a concentração em regiões específicas do espaço de soluções (Coello; Lamont; Van Veldhuizen, 2007; Deb *et al.*, 2002). A Figura 2 ilustra o funcionamento do algoritmo NSGA-II, utilizado na otimização de múltiplos objetivos.

Figura 2 – Diagrama de fluxo do NSGA-II



Fonte: Coello, Lamont e Van Veldhuizen (2007).

O processo inicia-se com a criação de uma população inicial de soluções (P_t) e outra de descendentes (Q_t), geradas por operadores genéticos como seleção, cruzamento e mutação. Essas populações são combinadas em uma única (R_t) e classificadas em frentes de Pareto (F_1 , F_2 , F_3 e assim por diante), de acordo com o grau de dominância entre as soluções.

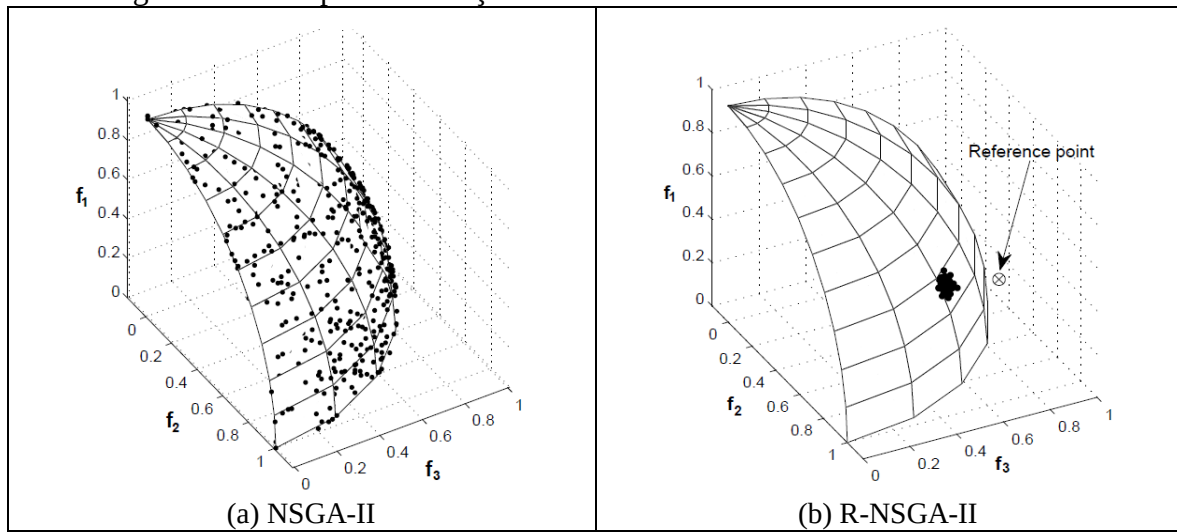
As melhores soluções são selecionadas a partir das frentes, priorizando a F_1 e, se necessário, complementadas pelas seguintes. Quando a adição de uma nova frente ultrapassa o

limite da população, aplica-se a métrica *crowding distance* para preservar a diversidade. Esse ciclo se repete a cada geração, permitindo que o algoritmo explore e refine continuamente o espaço de soluções, aproximando-se de um conjunto ótimo segundo múltiplos critérios.

Embora o NSGA-II apresente boa convergência e diversidade, ele possui limitações quando o objetivo é direcionar a busca para regiões específicas da fronteira de Pareto, pois sua estratégia de diversidade baseada em *crowding distance* tende a distribuir soluções de forma uniforme, sem considerar preferências do decisor (Deb *et al.*, 2002; Coello; Lamont; Van Veldhuizen, 2007). Para superar essa limitação, Deb e Sundar (2006), pesquisadores pioneiros em otimização evolucionária, propuseram o Reference Point Based Non-dominated Sorting Genetic Algorithm II (R-NSGA-II), uma extensão que introduz pontos de referência para direcionar o processo seletivo. Esses pontos permitem concentrar a busca em regiões aspiradas da fronteira, mas mantendo a diversidade local, tornando o algoritmo mais adequado para problemas em que há prioridades explícitas do tomador de decisão (Deb; Sundar, 2006).

A principal diferença operacional surge na etapa de seleção de sobrevivência. Após combinar pais e filhos e ordenar as soluções em frentes não-dominadas, o R-NSGA-II preenche a nova população com frentes completas exatamente como no NSGA-II, quando encontra uma frente parcial (cuja inclusão total excederia o tamanho da população), o algoritmo passa a ordenar as soluções dessa frente segundo sua proximidade aos pontos de referência fornecidos, em vez de usar apenas a *crowding distance* como critério de desempate (Deb; Sundar, 2006). Para cada solução da frente parcial calcula-se a distância normalizada até cada ponto de referência; soluções mais próximas recebem prioridade e são selecionadas primeiro, de modo que a população resultante fica concentrada nas regiões de interesse (Deb; Sundar, 2006; Coello; Lamont; Van Veldhuizen, 2007). Na Figura 3 é possível verificar a diferença das soluções não dominadas de um problema com o NSGA-II, pelo item a, e o R-NSGA-II, pelo item b, tornando-as mais concentradas no ponto de referência.

Figura 3 – Exemplo das soluções não dominadas do NSGA-II e R-NSGA-II

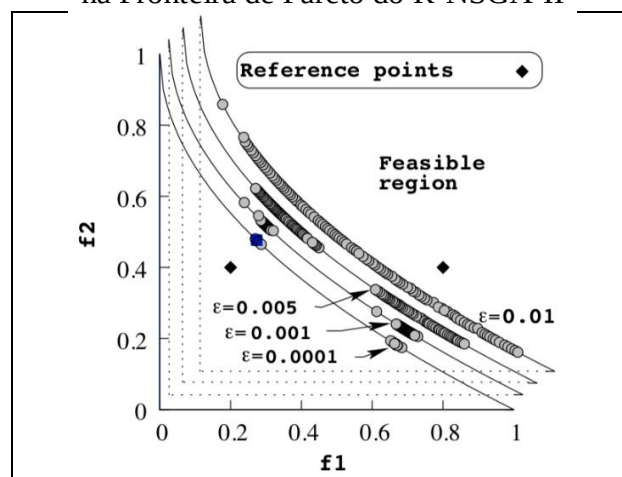


Fonte: Filatovas *et al.* (2015).

Adicionalmente, o R-NSGA-II costuma empregar um procedimento de filtragem local (*clustering*) controlado por um parâmetro ε ao selecionar a melhor solução relativa a um ponto de referência, o algoritmo remove temporariamente outras soluções que estejam dentro de uma vizinhança de raio ε dessa solução, evitando seleção redundante de pontos muito próximos e mantendo um espaçamento local adequado. A escolha de ε , bem como a estratégia de normalização dos objetivos e a definição dos pontos de referência, afeta diretamente a cobertura local e a diversidade das soluções obtidas, devendo ser alvo de análise de sensibilidade em experimentos empíricos (Deb *et al.*, 2002; Coello; Lamont; Van Veldhuizen, 2007).

A Figura 4 ilustra graficamente esse mecanismo, em que soluções mais próximas ao ponto de referência são priorizadas, enquanto o parâmetro ε controla a densidade local das soluções. Como mostrado, valores menores de ε concentram mais pontos ao redor da aspiração, enquanto valores maiores promovem maior espalhamento dentro da região de interesse.

Figura 4 – Demonstração de como o ponto de referência altera o comportamento das soluções na Fronteira de Pareto do R-NSGA-II



Fonte: Deb e Jayavelmurugan (2006).

O pseudocódigo do Quadro 1 apresenta de forma resumida o fluxo operacional do R-NSGA-II, destacando as etapas de inicialização, reprodução, ordenação não-dominada e seleção orientada por pontos de referência. Ele sintetiza como o algoritmo organiza o ciclo evolutivo e realiza a escolha das soluções que compõem a população a cada geração.

Quadro 1 – Pseudocódigo código do R-NSGA-II

```

Begin
  initialize the parameters popsize, maxiter, Z and eps where
  popsize: population size,
  maxiter: maximum number of iterations,
  Z: set of reference points,
  eps: neighborhood radius for clustering;
  generate initial population Pt with popsize individuals and evaluate objectives;

  t = 0;
  while t < maxiter

    # Reproduction
    select parents from Pt;
    generate offspring Qt by crossover and mutation;
    evaluate Qt;

    # Combine and non-dominated sorting
    Rt = Pt union Qt;
    perform non-dominated sorting on Rt (obtain fronts F1, F2, ...);

    # Build new population
    initialize Pnew as empty;
    insert full fronts into Pnew until the next front Fi would exceed popsize;
    if size(Pnew) < popsize then
      F = Fi # partial front

      # Reference-point association
      for each solution s in F
        for each reference point z in Z
          compute normalized distance d(s,z);
        end for
        assign rank_ref(s) = min_z d(s,z);
      end for

      # Reference-guided selection with epsilon-clearing
      while size(Pnew) < popsize
        select s_star in F with smallest rank_ref;
        insert s_star into Pnew;
        remove from F all solutions s with distance(s, s_star) <= eps;
      end while
    end if

    Pt = Pnew;
    t = t + 1;
  end while
  return Pt;
End

```

Fonte: adaptado de Filatovas *et al.* (2015).

No pseudocódigo identificam-se três estágios principais: (i) inicialização, representada pela definição de parâmetros no trecho Initialize the parameters popsize, maxiter, Z and eps where popsize: population size e pela geração e avaliação da população inicial Pt; (ii) processo evolutivo, implementado pelo laço while t < maxiter, no qual o algoritmo gera descendentes por seleção, crossover e mutação, combina pais e filhos em Rt = Pt union Qt, e organiza as soluções em frentes não-dominadas F1, F2, ..., permitindo o avanço gradual em direção à fronteira de Pareto; e (iii) seleção orientada por preferência, onde

a frente parcial P_i é tratada por meio da associação das soluções aos pontos de referência z , cálculo do $rank_ref$ e aplicação do ε -clearing para evitar concentrações excessivas ao redor de um mesmo ponto. Esse procedimento controla a diversidade local e garante que P_t seja atualizado com soluções alinhadas às regiões aspiradas, resultando ao final na população P_t concentrada nas áreas de maior interesse para o decisor.

2.3 TRABALHOS CORRELATOS

Para fundamentar o desenvolvimento deste trabalho, realizou-se uma revisão sistemática da literatura, com foco em estudos que aplicam técnicas de inteligência artificial, em especial algoritmos genéticos e modelos de aprendizado de máquina, na construção e personalização de carteiras de investimento. A pesquisa foi conduzida por meio de plataformas acadêmicas e bibliotecas digitais, utilizando filtros relacionados ao tema central deste projeto: a recomendação personalizada de portfólios financeiros com base no perfil do investidor e suas restrições.

Os filtros utilizados nas buscas foram formados pelas combinações: (“algoritmo genético” OR “alocação de portfólio”) AND (“perfil de investidor” OR “recomendação personalizada”) AND (“inteligência artificial” OR “sistema especialista”); (“IA” OR “machine learning”) AND (“mercado financeiro” OR “trading”) AND (“tomada de decisão” OR “otimização”).

Quadro 2 – Resultado de buscas por termos contidos em artigos em língua inglesa

Termos de busca	Litmaps	Sciente Direct	Google Acadêmico	IEEE Xplore	Total
"portfolio optimization" AND "investor profile" AND "artificial intelligence"	111	14	111	-	236
"portfolio optimization" AND "multi-objective algorithm" AND "investor profile"	2	10	2	-	14
"asset allocation" AND "genetic algorithm" AND "multi-objective optimization"	569	73	569	2	1.213
"investment strategy" AND "evolutionary algorithm" AND "artificial intelligence"	412	62	412	1	887
"financial decision" AND "machine learning" AND "portfolio recommendation"	45	8	45	-	98
"investment recommendation" AND "artificial intelligence" AND "risk-return"	85	10	85	-	180
"portfolio optimization" AND "deep learning" AND "asset diversification"	60	-	60	-	120
"portfolio management" AND "multi-objective optimization" AND "AI"	1.040	33	1.040	1	2.113
"AI investing" AND "genetic algorithm" AND "recommendation"	2	-	3	-	5
"financial portfolio" AND "algorithmic trading" AND "optimization model"	97	5	97	-	199

Fonte: elaborado pelos autores.

Inicialmente, realizou-se uma triagem para reduzir o número de artigos encontrados, que ultrapassava 2.000 resultados brutos, para um conjunto mais gerenciável de pouco mais de 250 estudos relevantes. Esse processo de filtragem levou em consideração a quantidade de citações, a presença de termos-chave no título e resumo. Foram excluídos artigos voltados a áreas não relacionadas, como ciências biológicas, sistemas logísticos ou algoritmos aplicados fora do contexto financeiro. Também foram descartados trabalhos puramente teóricos, sem aplicação prática em cenários de investimentos. Foram priorizados os estudos que abordavam algoritmos evolutivos (como o NSGA-II), sistemas de apoio à decisão financeira, e o uso de aprendizado de máquina na personalização de carteiras de ativos. Dessa forma, o conjunto final de trabalhos selecionados serviu de base para a fundamentação teórica e técnica do presente projeto.

Quadro 3 – Síntese dos trabalhos correlatos selecionados

Assunto	Filtro	Referência
Aplicações da inteligência artificial na análise e recomendação de investimentos	"inteligência artificial" AND "recomendação de investimentos" AND "perfil de risco"	Garcez (2022)
Inteligência artificial aplicada ao mercado financeiro: conceitos, aplicações e desafios	"inteligência artificial" AND "mercado financeiro"	Pavão (2023)
Inteligência artificial e investimentos: como a IA está transformando o trading	"inteligência artificial" AND "trading" AND "investimentos"	Slack (2024)
A inteligência artificial aplicada ao mercado de capitais: o uso de algoritmos no processo decisório de investimentos	"inteligência artificial" AND "processo decisório de investimentos"	Públio (2022)
Aplicação de algoritmos genéticos na alocação de portfólios de investimentos	"algoritmo genético" AND "carteira de investimentos" AND "multiobjetivo"	Guarino (2024)

Fonte: elaborado pelos autores.

Os trabalhos apresentados no Quadro 3 foram selecionados por abordarem, direta ou indiretamente, os principais elementos que compõem a proposta deste trabalho: algoritmos genéticos, recomendação personalizada de investimentos e o uso de inteligência artificial na tomada de decisões financeiras.

O trabalho de Guarino et al. (2024) propôs o *EvoFolio*, um método de otimização de portfólios baseado em algoritmos evolutivos multiobjetivo, especificamente o NSGA-II, que busca maximizar o retorno e minimizar o risco simultaneamente. Utilizando uma base de dados com 47 ativos em um período de alta volatilidade (2020-2022), os autores implementam uma abordagem inspirada na "*Human-Centered AI*", permitindo que investidores insiram suas preferências de mercado ("*views*") para guiar o processo de seleção através de elitismo iterativo. O estudo demonstra que a incorporação dessas preferências reduz significativamente o risco das carteiras e, em comparações diretas, supera o desempenho financeiro de agentes baseados em Aprendizado por Reforço.

Garcez (2022) comparou a eficácia de dois métodos de previsão de séries temporais aplicados ao mercado financeiro: o modelo estatístico ARIMA e a técnica de Programação Genética (PG), uma vertente da Inteligência Artificial Evolucionária. Utilizando dados reais dos índices IBOVESPA, NASDAQ e Dow Jones, o autor aplicou ambos os métodos para prever valores futuros com base em séries históricas. Os resultados demonstraram que, nas primeiras previsões ex-post, a PG apresentou desempenho superior ao ARIMA em termos de precisão. A conclusão do estudo sugere que, embora os modelos estatísticos tradicionais sejam robustos, técnicas de IA como a PG oferecem maior flexibilidade e capacidade de adaptação em cenários financeiros complexos e voláteis.

Pavão (2023) apresenta uma aplicação prática da IA na previsão de preços de ações, utilizando o método de suavização exponencial tripla (ETS) com dados da PETR4. Com abordagem exploratória e uso da linguagem Python, o autor demonstra como algoritmos estatísticos simples podem auxiliar investidores a identificar tendências e tomar decisões mais informadas, mesmo sem conhecimento técnico avançado. O estudo conclui que ferramentas acessíveis baseadas em IA podem democratizar o uso de análise preditiva no mercado financeiro, tornando-as úteis para o investidor comum.

Slack (2025) explora diversas técnicas, incluindo *trading* algorítmico, negociação de alta frequência (HFT), análise preditiva, análise de sentimento, monitoramento de mercado, filtros de seleção de ações, robôs assessores, gestão de risco e otimização de carteiras. Ferramentas como o InvestingPro são mencionadas como recursos que utilizam IA para auxiliar investidores na escolha de ações brasileiras, com estratégias baseadas em *backtests* de 10 anos.

O trabalho de Públio (2022) investigou como a inteligência artificial está sendo utilizada na gestão financeira de empresas brasileiras. A metodologia incluiu uma pesquisa de campo com 10 profissionais do setor financeiro, além de revisão de literatura sobre as aplicações da IA em análise de dados, detecção de fraudes, avaliação de riscos e automação de processos. Os resultados revelam que 80% dos entrevistados consideram a IA extremamente importante, e 70% relatam benefícios concretos como redução de custos, otimização de processos e apoio à tomada de decisão. A conclusão destaca que, embora a IA represente um diferencial competitivo crescente, sua implementação requer atenção a questões como segurança de dados, ética e capacitação das equipes.

3 DESENVOLVIMENTO

Para um melhor entendimento do desenvolvimento da aplicação, este capítulo foi subdividido em duas seções principais. Na seção 3.1, será abordado o desenvolvimento do modelo de otimização de portfólios (*backend*). Já a seção 3.2 apresentará a arquitetura, funcionalidades e principais aspectos da plataforma web, nomeada como WalletAI, para apoiar investidores de diferentes níveis de experiência na tomada de decisões (*frontend*).

3.1 MODELO DE OTIMIZAÇÃO DE PORTFÓLIOS (BACKEND)

O desenvolvimento do *backend* constitui o núcleo computacional da plataforma de otimização de portfólios, integrando módulos para ingestão e tratamento de dados históricos, execução do algoritmo de otimização e exposição de resultados por meio de uma API REST. Essa arquitetura modular garante a separação de responsabilidades, reduz o acoplamento entre componentes e facilita manutenção, testes e evolução da solução. O trabalho de desenvolvimento inicia-se com o levantamento de requisitos, que orienta as escolhas de implementação e operação.

3.1.1 Requisitos

O modelo foi desenvolvido atendendo os seguintes Requisitos Funcionais (RF) e Requisitos Não Funcionais (RNF):

- a) autenticar usuários através de credenciais seguras (email e senha) e gerar *tokens* JWT para manutenção de sessões autorizadas (RF);
- b) coletar dados históricos mensais de preços via API Yahoo Finance e armazenar séries temporais com cálculo de variações percentuais (RF);
- c) otimizar carteiras através do algoritmo R-NSGA-II considerando maximização de retorno, minimização de variância e minimização de CVaR (RF);
- d) aplicar perfis de risco personalizados (conservador, moderado e arrojado), restrições de ativos específicos e limitação na quantidade de ativos da carteira durante o processo de otimização (RF);
- e) realizar simulação de *backtest* comparando performance histórica das carteiras otimizadas com *benchmark* de mercado (RF);
- f) utilizar linguagem Python com *framework* Flask para API REST (RNF);
- g) utilizar o PostgreSQL como banco de dados relacional (RNF);
- h) utilizar a biblioteca `pymoo` para realizar o processamento do AGMO (RNF);
- i) ser capaz de realizar a otimização em um tempo máximo de um minuto (RNF);

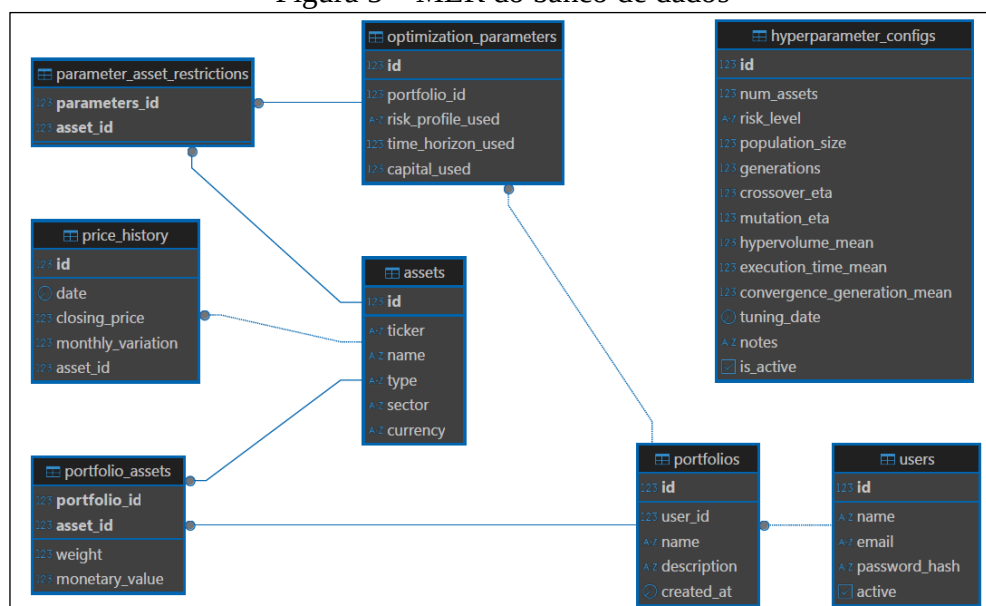
3.1.2 Modelagem do banco de dados

Optou-se por utilizar o PostgreSQL como sistema gerenciador de banco de dados responsável pelo armazenamento das informações relacionadas aos usuários, às operações do sistema e aos resultados gerados pelos módulos de processamento. O PostgreSQL é um Sistema de Gerenciamento de Banco de dados (SGBD) relacional de código aberto, amplamente reconhecido pela sua confiabilidade, robustez e aderência a padrões SQL, além de oferecer recursos avançados que o colocam no mesmo patamar de soluções corporativas utilizadas em larga escala.

A escolha por um banco de dados relacional fundamenta-se na necessidade de manter consistência, integridade e estruturação rigorosa das informações manipuladas pela aplicação. Em especial, o modelo relacional favorece o uso de esquemas bem definidos, com relacionamentos claros entre entidades, o que é essencial em sistemas que lidam com dados estruturados, como cadastros, registros históricos, configurações de execução e metadados utilizados no processo de otimização de carteiras. Esse formato garante maior segurança na manipulação dos dados, facilita auditorias e reduz ambiguidades durante o processamento.

No contexto deste projeto, tais características foram fundamentais para dar suporte a um ambiente de processamento intensivo, garantindo organização, segurança dos dados, eficiência nas consultas e escalabilidade conforme o volume de avaliações das carteiras cresce. A Figura 5 apresenta o Modelo Entidade-Relacionamento (MER) desenvolvido para estruturar as tabelas responsáveis pelo gerenciamento dos usuários, ativos financeiros, parâmetros do algoritmo e resultados da otimização.

Figura 5 – MER do banco de dados



Fonte: elaborado pelos autores.

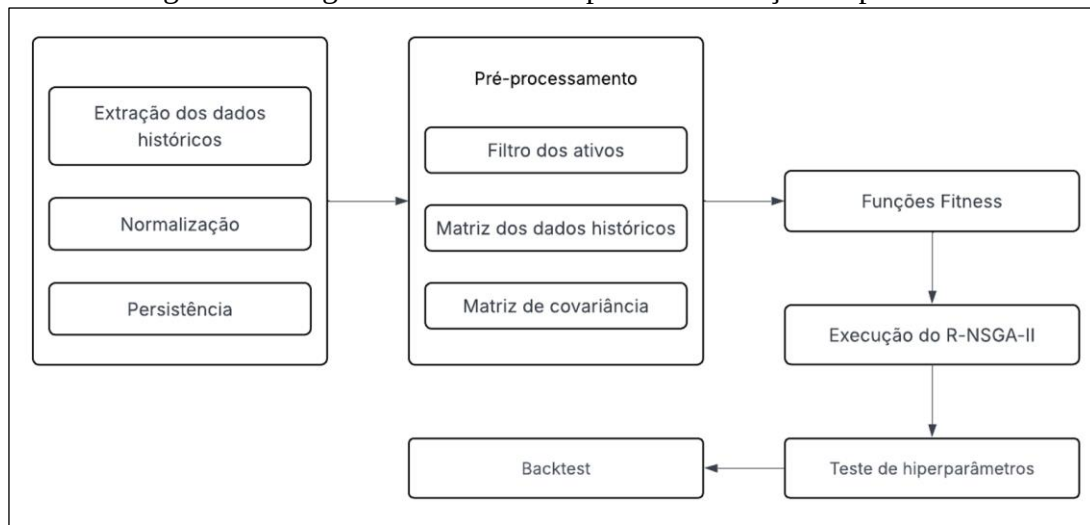
Com base no MER da Figura 5, é possível identificar os principais componentes que estruturam o banco de dados e a forma como se conectam. A seguir, são detalhadas cada uma das tabelas derivadas desse modelo, juntamente com suas funções e papéis dentro da arquitetura geral do banco de dados.

- a) `users`: armazena informações de autenticação e identificação, incluindo *hash* seguro da senha. Esta tabela relaciona-se com carteiras através de uma associação um-para-muitos, permitindo que cada usuário mantenha múltiplas carteiras otimizadas
- b) `assets`: centraliza informações sobre instrumentos financeiros disponíveis para otimização. Cada ativo possui *ticker* único, nome descritivo, classificação por tipo e setor econômico. Esta estrutura facilita filtragens e agrupamentos durante o processo de seleção de ativos;
- c) `price_history`: armazena a série temporal dos preços de fechamento mensais. Esta desnormalização controlada otimiza consultas frequentes durante o processo de otimização, evitando consultas externas repetitivas;
- d) `portfolios`: armazena as informações descritivas da carteira, guarda a composição final dos ativos gerados pela otimização
- e) `optimization_parameters`: preserva o contexto completo utilizado em cada otimização, incluindo perfil de risco, horizonte temporal e limite de ativos
- f) `hyperparameter_configs`: armazena as configurações ótimas de hiperparâmetros determinadas através de processos sistemáticos de *tuning*. Cada registro mapeia uma quantidade específica de ativos para valores ideais de tamanho populacional e número de gerações, permitindo que o sistema ajuste automaticamente seus parâmetros baseado na complexidade do problema.

3.1.3 Diagrama de atividades

O diagrama de atividades da Figura 6 apresenta as etapas seguidas no desenvolvimento do *backend*, destacando as ações principais e a sequência lógica que orienta a construção do modelo de otimização de portfólio.

Figura 6 – Diagrama de atividades para a otimização de portfólio



Fonte: elaborado pelos autores.

Primeiramente é executado a extração e validação das séries históricas, que são tratadas e reorganizadas por meio de operações de pivotamento, preenchimento de lacunas, cálculo dos retornos e exclusão de ativos com histórico insuficiente. A partir desses dados limpos, são calculadas as estatísticas necessárias ao processo de otimização, incluindo o vetor de retornos médios e a matriz de covariância. Com essas informações, o algoritmo AGMO é executado, gerando populações iniciais, aplicando operadores especializados que respeitam restrições de cardinalidade e conduzindo o processo evolutivo por meio da seleção baseada em pontos de referência. Para saber qual é o valor dos hiperparâmetros adequados para um determinado número de ativos é feito várias análises comparando o indicador R-Hypervolume. Na sequência, realiza-se o *backtest* das carteiras recomendadas, comparando seu desempenho histórico com *benchmarks* representativos dos perfis conservador e arrojado.

3.1.4 Extração de dados

O processo de extração de dados financeiros constitui etapa fundamental para garantir a qualidade das otimizações subsequentes. Inicialmente, deve-se baixar um arquivo `.csv` com todos os ativos disponíveis, conforme ilustra a Figura 7.

Figura 7 – Arquivo `.csv` de ativos

```

ticker,nome,tipo,setor
ALOS3,ALOS,STOCK,Saúde
ABEV3,AMBEV S/A,STOCK,Consumo
ANIM3,ANIMA,STOCK,Educação
ASAI3,ASSAI,STOCK,Varejo
AURE3,AUREN,STOCK,Energia Elétrica
AXIA3,AXIA ENERGIA,STOCK,Energia Elétrica
AXIA6,AXIA ENERGIA,STOCK,Energia Elétrica
AZZA3,AZZAS 2154,STOCK,Varejo
B3SA3,B3,STOCK,Financeiro
BBSE3,BBSEGURIDADE,STOCK,Seguros
  
```

Fonte: elaborado pelos autores.

Para os testes desse projeto foram utilizados os 100 ativos mais líquidos da B3 utilizando o índice IBX100, referência amplamente utilizada no mercado para representar ações com maior volume e relevância. O Quadro 4 apresenta o trecho de código responsável por armazenar os ativos.

Quadro 4 – Trecho do código para persistência dos ativos

```
reader = csv.DictReader(f)
for row in reader:
    asset = Asset.query.filter_by(ticker=row['ticker']).first()
    if not asset:
        new_asset = Asset(
            ticker=row['ticker'],
            name=row['nome'],
            type=row['tipo'],
            sector=row.get('setor')
        )
        db.session.add(new_asset)
db.session.commit()
```

Fonte: elaborado pelos autores.

Uma vez armazenado o conjunto de ativos, o sistema inicia a coleta das séries históricas por meio da biblioteca *yfinance*, que estabelece conexão direta com os servidores do Yahoo Finance. Para cada ativo previamente cadastrado, o sistema obtém o histórico completo no intervalo mensal, garantindo que os preços retornados estejam devidamente ajustados por dividendos e desdobramentos. O Quadro 5 apresenta o processo de extração dos dados no *yfinance* para a ação BBAS3, retornando um arquivo com a data e o preço de fechamento do ativo.

Quadro 5 – Trecho do código para a extração dos dados do *yfinance*

```
def _download_data(self, ticker, interval, period=None, start=None, end=None):
    try:
        params = {
            'interval': interval,
            'progress': False,
            'auto_adjust': True # ajustado por dividendos e desdobramento
        }

        if period:
            params['period'] = period
        else:
            if start:
                params['start'] = start
            if end:
                params['end'] = end

        return yf.download(ticker + '.SA', **params)
    except Exception as e:
        print(f" - Erro ao baixar dados: {e}")
        return pd.DataFrame()

def process(self, asset):
    period = "max"
    try:
        # Download de dados mensais históricos
        data = self._download_data(asset.ticker, interval="1mo", period=period)
```

Fonte: elaborado pelos autores.

Após a obtenção das séries históricas, calcula-se a variação percentual mensal de cada ativo, a partir da diferença relativa entre os preços de fechamento. Essa métrica constitui um

dos elementos fundamentais para a construção das matrizes estatísticas empregadas no modelo multiobjetivo. Na sequência, cada registro mensal é avaliado individualmente e comparado ao conteúdo já existente no banco de dados. Apenas valores inéditos são persistidos no banco de dados, evitando duplicidades e garantindo consistência no histórico armazenado. O Quadro 6 mostra um exemplo do *DataFrame* utilizado, contendo a data, o preço de fechamento do ativo nessa data e a variação calculada dos preços

Quadro 6 – *DataFrame* de um período de 6 meses da ação BBAS3

Date	Close	Variation
2000-01-01	0.43	
2000-02-01	0.40	-0.056963
2000-03-01	0.34	-0.155704
2000-04-01	0.33	-0.046104
2000-05-01	0.30	-0.066667
2000-06-01	0.35	0.151786

Fonte: elaborado pelo autor.

Complementando esse fluxo, desenvolveu-se um agendador noturno responsável por manter o banco de dados constantemente atualizado, conforme demonstrado no Quadro 7. Esse agendador é executado diariamente às 23 horas e tem como função realizar a extração dos preços mais recentes disponíveis para cada ativo monitorado, utilizando a mesa rotina que a extração completa. Assim que os novos dados são obtidos, o sistema repete o processo de validação e persistência, inserindo apenas os valores ainda não registrados. Com isso, o histórico financeiro permanece atualizado de forma automática, permitindo que as otimizações realizadas posteriormente utilizem dados recentes e confiáveis, sem necessidade de intervenções manuais.

Quadro 7 – Trecho de código da execução do agendador

```
def start(self, hour=23, minute=0):
    if self.scheduler.running:
        logger.warning("Scheduler já está em execução.")
        return

    # Adicionar job com trigger cron para executar diariamente
    trigger = CronTrigger(hour=hour, minute=minute, timezone='America/Sao_Paulo')

    self.scheduler.add_job(
        func=self._job_wrapper,
        trigger=trigger,
        id='daily_price_update',
        name='Atualização Diária de Preços',
        replace_existing=True
    )

    self.scheduler.start()
```

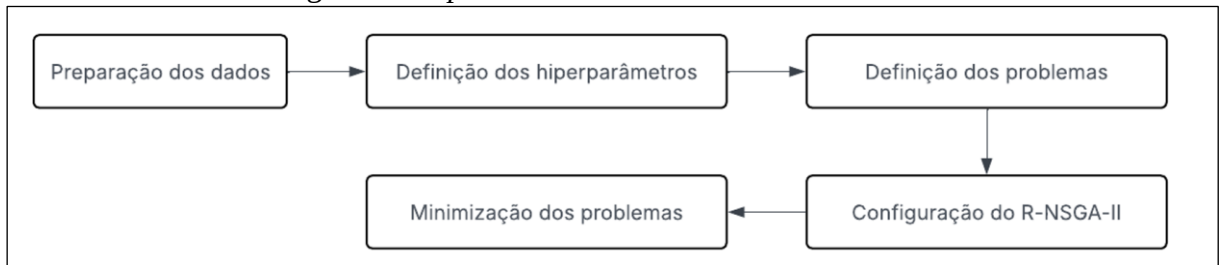
Fonte: elaborado pelos autores.

3.1.5 Implementação do AGMO

A implementação do algoritmo genético multiobjetivo constitui o núcleo computacional da plataforma de otimização, integrando conceitos avançados de computação evolutiva com teoria moderna de portfólios. O desenvolvimento seguiu uma abordagem modular, dividindo

as responsabilidades do processamento, conforme *pipeline* apresentado na Figura 8. Cada uma dessas etapas será detalhada nas subseções a seguir.

Figura 8 – *Pipeline* de desenvolvimento do AGMO



Fonte: elaborado pelos autores.

3.1.5.1 Preparação dos dados

A preparação dos dados é uma das etapas mais determinantes no processo de otimização de portfólios, pois toda a análise de risco e retorno depende diretamente da qualidade das séries históricas utilizadas. Esse papel é desempenhado pelo método `_prepare_data` (Quadro 8), responsável por transformar os dados brutos armazenados no banco em um conjunto estruturado e estatisticamente confiável para o algoritmo genético multiobjetivo.

Quadro 8 – Trecho de código da preparação dos dados.

```

def _prepare_data(self):
    with self.app.app_context():
        assets_query = db.session.query(Asset).filter(
            ~Asset.id.in_(self.restricted_asset_ids),
            Asset.type == AssetType.STOCK
        )
        if self.asset_ids:
            assets_query = assets_query.filter(Asset.id.in_(self.asset_ids))

        self.assets_to_optimize = assets_query.all()

        ids_to_optimize = [a.id for a in self.assets_to_optimize]
        history_query = db.session.query(
            PriceHistory.date,
            PriceHistory.monthly_variation,
            Asset.ticker
        ).join(Asset, PriceHistory.asset_id == Asset.id) \
            .filter(PriceHistory.asset_id.in_(ids_to_optimize))

        history_query = history_query.order_by(PriceHistory.date)

        df_history = pd.read_sql(
            history_query.statement,
            con=db.session.connection()
        )

        df_returns_complete = df_history.pivot(
            index='date',
            columns='ticker',
            values='monthly_variation'
        )
  
```

Fonte: elaborado pelos autores.

O método inicia consultando os ativos elegíveis e seus metadados, definindo assim o universo de investimento que será utilizado na otimização, em seguida, são carregados os históricos de preços de cada ativo, conforme exemplifica a Quadro 9.

Quadro 9 – Exemplo das linhas retornadas do banco de dados

Date	Ticker	Variation
2020-01-01	BBAS3	-0.124659
2020-01-01	PETR4	-0.058221
2020-01-01	VALE3	0.005718
2020-02-01	BBAS3	-0.171589
2020-02-01	PETR4	-0.166729
2020-02-01	VALE3	0.074701

Fonte: elaborado pelos autores.

Esses dados, são reorganizados em um *DataFrame* pivotado no qual cada coluna representa um *ticker* e cada linha representa uma data (Quadro 10). Essa estrutura matricial é essencial para o cálculo de retornos e para todas as operações vetorizadas que compõem o modelo de risco.

Quadro 10 – Exemplo da transformação de dados para a matriz

Date	BBAS3	PETR4	VALE3
2020-01-01	-0.124659	-0.058221	0.005718
2020-02-01	-0.171589	-0.166729	0.074701

Fonte: elaborado pelos autores.

Após o pivotamento, o método identifica o período mínimo de dados exigido, sendo igual ao período solicitado pelo usuário como horizonte de investimentos (Quadro 11). Ativos que não atendem essa exigência são removidos. Essa decisão tem como objetivo igualar todos os ativos com o mesmo período de dados históricos, para não ter injustiças na comparação entre eles.

Quadro 11 – Filtro com base no horizonte de investimento

```
# Filtrar ações antes do pivot baseado no horizonte de investimento
minimum_history_months = int(self.years_period * 12)

for ticker in available_assets:
    available_months = data_count[ticker]

    if available_months >= minimum_history_months:
        valid_assets.append(ticker)
    else:
        excluded_assets.append(ticker)

# Filtra o DataFrame original para incluir apenas ativos válidos
df_history_filtered = df_history[df_history['ticker'].isin(valid_assets)]
```

Fonte: elaborado pelos autores.

Com a filtragem inicial concluída, é gerado duas estruturas essenciais para a otimização: (i) o retorno médio e (ii) a matriz de covariância (Quadro 12). O vetor `mean_returns` é criado a partir da média dos retornos de cada ativo, representando seu desempenho esperado no período. Em seguida, a matriz de covariância é calculada com `df_returns.cov()`. Esses dois elementos são armazenados no banco de dados para uso do algoritmo, servindo de base para avaliar risco e retorno de cada solução candidata.

Quadro 12 – Cálculo das médias de retorno e matriz de covariância.

```
# Calcular estatísticas
self.mean_returns = df_returns.mean()
self.covariance_matrix = df_returns.cov()
```

Fonte: elaborado pelos autores.

A matriz de covariância é calculada diretamente a partir da matriz de retornos históricos e captura tanto a volatilidade individual dos ativos quanto a forma como seus retornos variam conjuntamente. Partindo da matriz R , obtém-se o vetor de médias μ e, em seguida, a matriz de retornos centrados x . O estimador amostral da covariância é dado pelas fórmulas da Equação 3.

$$\Sigma = \frac{1}{T-1} D^T D, \quad \text{Cov}(i, j) = \frac{1}{T-1} \sum_{t=1}^T (r_{t,i} - \mu_i)(r_{t,j} - \mu_j), \quad (3)$$

Onde T é o número de observações, r o retorno do ativo i no tempo t e μ sua média. Os elementos da diagonal representam as variâncias, indicando a volatilidade individual de cada ativo, enquanto os elementos fora da diagonal mensuram como os ativos se movimentam juntos. Por exemplo, no Quadro 13 observa-se que PETR4 apresenta a maior variância (0,013272), refletindo sua elevada volatilidade histórica. A covariância entre BBAS3 e ITUB4 (0,007104) é relativamente alta, indicando que ambos tendem a se mover na mesma direção, compatível com o fato de pertencerem ao mesmo setor financeiro. Já a covariância entre WEGE3 e VALE3 (0,001222) é significativamente menor, evidenciando que seus retornos apresentam comovimento fraco, o que contribui para a diversificação do portfólio. Essa interpretação combinada permite compreender quais pares de ativos adicionam risco conjunto e quais ajudam a reduzi-lo, sendo a base para o cálculo do risco total do portfólio.

Quadro 13 – Matriz de covariância de 5 ações

	BBAS3	ITUB4	PETR4	VALE3	WEGE3
BBAS3	0.013312	0.007104	0.008844	0.003650	0.002108
ITUB4	0.007104	0.007615	0.005863	0.002861	0.001790
PETR4	0.008844	0.005863	0.013272	0.004799	0.001726
VALE3	0.003650	0.002861	0.004799	0.009879	0.001222
WEGE3	0.002108	0.001790	0.001726	0.001222	0.007556

Fonte: elaborado pelos autores.

3.1.5.2 Formulação multiobjetivo do problema

A modelagem do problema de otimização de portfólios adota formulação tri-objetivo que captura *trade-offs* fundamentais enfrentados por investidores racionais. A classe `PersonalizedPortfolioProblem` estende da interface `ElementwiseProblem` da biblioteca `pymoo` (Quadro 14), implementando avaliação simultânea de três funções objetivo conflitantes.

Quadro 14 – Cálculos das funções de *Fitness*

```
def _evaluate(self, x, out, *args, **kwargs):
    weights = x

    # Obj 1: Retorno esperado (negativo porque o pymoo minimiza)
    expected_return = -np.dot(weights, self.mu)

    # Obj 2: Risco (variância)
    variance = np.dot(weights, self.cov @ weights)

    # Obj 3: Risco de cauda (CVaR)
    cvar = self._calculate_cvar(weights)

    out["F"] = [expected_return, variance, cvar]
```

Fonte: elaborado pelos autores.

O primeiro objetivo busca maximizar o retorno esperado do portfólio, usando a média dos retornos diários calculados na preparação dos dados. Como a biblioteca `pymoo` é projetado para problemas de minimização, a função objetivo de retorno esperado foi multiplicada por -1. Dessa forma, a minimização do 'retorno negativo' equivale matematicamente à maximização do retorno absoluto.

O segundo objetivo minimiza a variância do portfólio, o cálculo considera matriz completa de covariâncias. Esta formulação permite identificação de oportunidades de diversificação onde correlações negativas ou baixas reduzem risco total do portfólio.

O terceiro objetivo minimiza o Conditional Value at Risk (CVaR), uma métrica de risco que quantifica a perda média nos cenários mais extremos da distribuição (Quadro 15). Essa medida estima quanto a carteira tende a perder quando o mercado já se encontra em condições adversas, capturando a porção de maior risco da cauda. Neste trabalho, o CVaR é calculado de forma empírica a partir dos retornos históricos: após aplicar os pesos da solução sobre a matriz de retornos, obtêm-se as perdas da carteira e, a partir delas, o CVaR corresponde à média dos piores $\alpha \times n$ valores. O parâmetro α define a fração da cauda analisada e tem o valor fixo de 5%.

Quadro 15 – Cálculo do CVaR

```
def _calculate_cvar(self, weights):
    # 1. Calcular retornos e perdas da carteira
    portfolio_returns = self.hist @ weights
    losses = -portfolio_returns

    # 2. Filtrar valores inválidos
    valid_losses = losses[np.isfinite(losses)]
    n = len(valid_losses)

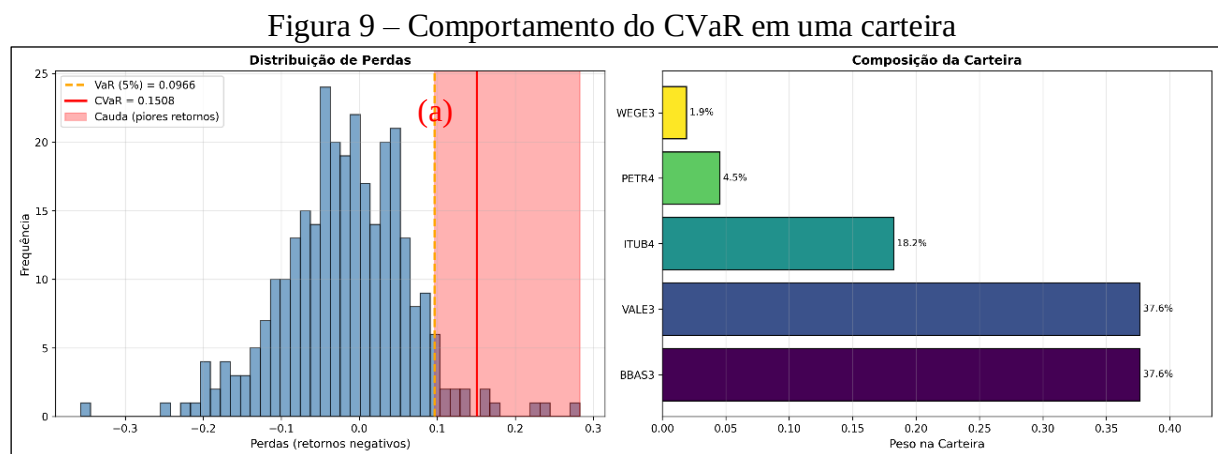
    # 3. Calcular número de observações na cauda
    k = max(1, int(np.ceil(self.alpha * n)))

    # 4. CVaR = média dos k piores retornos (maiores perdas)
    sorted_losses = np.sort(valid_losses)
    tail = sorted_losses[-k:] # Sempre exatamente k observações

    return float(np.mean(tail))
```

Fonte: elaborado pelos autores.

A Figura 9 ilustra o comportamento do CVaR no contexto das carteiras avaliadas pelo algoritmo. No painel esquerdo, apresenta-se a distribuição das perdas históricas da carteira selecionada, onde a região destacada em vermelho corresponde à cauda superior, isto é, aos piores cenários observados na série. A linha tracejada (item a) indica o ponto de corte que separa a fração de 5%, enquanto o valor do CVaR representa a média das perdas dentro dessa área sombreada, refletindo o quanto a carteira tende a perder quando ocorrem eventos adversos de maior severidade. Já o painel direito mostra a composição da carteira utilizada para gerar essa distribuição, permitindo visualizar como os pesos atribuídos a cada ativo influenciam o risco extremo.



Fonte: elaborado pelos autores.

3.1.5.3 Implementação do algoritmo R-NSGA-II

Para resolver o problema multiobjetivo foi usado o algoritmo R-NSGA-II no contexto da otimização de portfólios de investimento, pois investidores possuem perfis de risco distintos e preferências individuais que influenciam suas decisões de alocação de recursos. Enquanto o NSGA-II tradicional gera toda a fronteira de Pareto de forma uniforme, sem considerar as preferências do investidor durante o processo de otimização, o R-NSGA-II permite que estas preferências sejam incorporadas desde o início, através da definição de pontos de referência que representam os objetivos desejados pelo investidor.

No sistema desenvolvido, os pontos de referência são definidos automaticamente com base no perfil de risco especificado pelo investidor (Quadro 16). Para investidores com perfil conservador, o sistema prioriza a minimização dos riscos, configurando um ponto de referência que atribui maior importância aos objetivos de variância e CVaR. Investidores com perfil moderado recebem pontos de referência balanceados entre retorno e risco, enquanto o perfil arrojado concentra-se na maximização de retorno, tolerando níveis mais elevados de risco.

Quadro 16 – Algoritmo R-NSGA-II

```

# Reference Points: onde queremos chegar (aspirações no espaço normalizado [0,1])
# - 0.0 = melhor valor possível (min risco / max retorno)
# - 1.0 = pior valor possível (max risco / min retorno)
REFERENCE_POINTS_CONFIG = {
    'conservador': np.array([[0.3, 0.05, 0.05]]),
    'moderado':    np.array([[0.3, 0.2, 0.2]]),
    'arrojado':    np.array([[0.05, 0.25, 0.25]])
}

WEIGHTS_CONFIG = np.array([0.33, 0.34, 0.33])

def get_algorithm(self, crossover_eta: float, mutation_eta: float,
                    population_size: int, max_assets: int = None) -> NSGA2:
    # Operadores customizados com restrição de cardinalidade
    sampling = SimplexSamplingCardConstraint(max_assets=max_assets)
    crossover = SimplexCrossoverCardConstraint(max_assets=max_assets, eta=crossover_eta)
    mutation = SimplexMutationCardConstraint(max_assets=max_assets, eta=mutation_eta)

    # Usa configuração centralizada (constantes do módulo)
    ref_points = REFERENCE_POINTS_CONFIG.get(self.risk_level)
    weights = WEIGHTS_CONFIG.get(self.risk_level)

    return RNSGA2(
        ref_points=ref_points,
        pop_size=population_size,
        crossover=crossover,
        mutation=mutation,
        sampling=sampling,
        epsilon=0.01,
        normalization='front',
        extreme_points_as_reference_points=False,
        weights=weights
    )

```

Fonte: elaborado pelos autores.

No contexto de algoritmos genéticos, cada indivíduo representa uma solução candidata; no problema de portfólio, ele corresponde a uma carteira de investimentos, cujos pesos dos ativos funcionam como genes. Um portfólio com N ativos é representado por um vetor $w = (w_1, \dots, w_n)$, onde cada w_i indica a fração do capital alocada no ativo i , sujeito à restrição fundamental de que a soma dos pesos seja igual a um ($\sum w_i = 1$). O método `enforce_cardinality` (Quadro 17) é responsável por tratar essa situação, identificando todos os ativos com peso superior a um limiar pequeno (10^{-6}), ordena-os em ordem decrescente de peso, preserva os K ativos com maiores pesos enquanto zera os demais, e renormaliza dividindo cada peso preservado pela soma de todos os pesos preservados. Esta abordagem garante que o portfólio resultante satisfaz a cardinalidade (número de ativos $\leq K$), baseando-se na premissa de que ativos com pesos maiores contribuem mais significativamente para os objetivos do portfólio.

Por exemplo, considere o vetor de pesos $[0.45, 0.10, 0.30, 0.05, 0.10]$ e uma cardinalidade desejada de 2. Os dois maiores valores são 0,45 e 0,30, portanto apenas eles são mantidos; os demais são zerados. Após a renormalização proporcional desses dois pesos, obtém-se $[0.60, 0, 0.40, 0, 0]$.

Quadro 17 – Implementação da função `enforce_cardinality`.

```
def _enforce_cardinality(weights: np.ndarray, max_assets: int) -> np.ndarray:
    n_var = len(weights)

    active_mask = weights > 1e-6
    n_active = np.sum(active_mask)

    # Se já está dentro do limite, retorna
    if n_active <= max_assets:
        return weights

    # Precisa reduzir: mantém top-K ativos por peso
    # Ordena índices por peso decrescente
    sorted_indices = np.argsort(-weights)

    # Cria novo vetor zerado
    new_weights = np.zeros(n_var)

    # Mantém apenas os top max_assets ativos
    for i in range(max_assets):
        idx = sorted_indices[i]
        new_weights[idx] = weights[idx]

    # Renormaliza para soma = 1
    weight_sum = new_weights.sum()
    if weight_sum > 0:
        new_weights = new_weights / weight_sum
    else:
        # Fallback: distribuição uniforme nos max_assets primeiros
        new_weights[sorted_indices[:max_assets]] = 1.0 / max_assets

    return new_weights
```

Fonte: elaborado pelos autores.

Além da restrição de cardinalidade, adotou-se um padrão de limitar todos os pesos dos ativos em um intervalo contínuo $[0,01, 0,30]$. O limite inferior evita alocações insignificantes que seriam impraticáveis devido a custos de transação, enquanto o limite superior impede concentração excessiva em um único ativo, promovendo diversificação mesmo quando o otimizador identifica ativos com desempenho superior. Para isso, utilizou-se o método `clip` da biblioteca `numpy`, que força cada componente do vetor de pesos a permanecer dentro do intervalo admissível, e depois renormaliza para manter a soma dos pesos igual a 1 (Quadro 18).

Quadro 18 – Implementação da limitação de pesos.

```
# Garantir limites do problema
weights = np.clip(weights, problem.xl, problem.xu)

# Re-normalizar após clip
weights = weights / weight_sum
```

Fonte: elaborado pelos autores.

Essas condições definem o simplex unitário n -dimensional e tornam inadequados os operadores genéticos tradicionais, que não consideram restrições interdependentes. Para garantir viabilidade em todas as gerações, este trabalho utiliza operadores especializados que produzem apenas portfólios válidos: `SimplexSamplingCardConstraint` para amostragem inicial, `SimplexCrossoverCardConstraint` para cruzamento, e

`SimplexMutationCardConstraint` para mutação. Dessa forma é possível chamar a implementação customizada de cardinalidade ao final de cada operação

O operador `SimplexSamplingCardConstraint` utiliza a distribuição de Dirichlet, uma distribuição de probabilidade multivariada definida sobre o *simplex*, isto é, o conjunto de todos os vetores formados por valores não negativos que, somados, resultam exatamente em 1. Esse espaço corresponde justamente ao domínio possível dos pesos de um portfólio, pois cada peso deve ser positivo e a soma total deve representar 100% do capital investido. Quando há restrição de cardinalidade K, o operador seleciona aleatoriamente K dos N ativos, aplica a Dirichlet com $\alpha = (1, \dots, 1)$ apenas sobre esses ativos e define peso zero para os demais (Quadro 19).

Quadro 19 – Implementação da geração inicial

```
def _do(self, problem, n_samples, **kwargs):
    n_var = problem.n_var

    X = np.zeros((n_samples, n_var))

    for i in range(n_samples):
        if self.max_assets is None or self.max_assets >= n_var:
            # Sem restrição de cardinalidade: usa Dirichlet padrão
            weights = np.random.dirichlet(np.ones(n_var))
        else:
            # Escolhe quais ativos estarão ativos (amostragem sem reposição)
            active_indices = np.random.choice(n_var, size=self.max_assets, replace=False)

            # Gera pesos usando Dirichlet apenas para ativos selecionados
            active_weights = np.random.dirichlet(np.ones(self.max_assets))

            # Cria vetor completo
            weights = np.zeros(n_var)
            weights[active_indices] = active_weights
```

Fonte: elaborado pelos autores.

Para ilustrar o funcionamento do operador, considere um universo hipotético com cinco ativos $N=5$ e uma restrição de cardinalidade igual a três $K=4$ (Quadro 20). Em uma geração inicial, o operador seleciona aleatoriamente três ativos para compor o portfólio

Quadro 20 – Exemplo da geração de uma população inicial

Ativo	Selecionado?	Peso gerado
BBAS3	Sim	0.45
PETR4	Não	0.00
VALE3	Sim	0.11
ITUB4	Sim	0.37
WEGE3	Sim	0.07

Fonte: elaborado pelo autor.

O operador `SimplexCrossoverCardConstraint` adapta o SBX ao contexto de portfólios, garantindo que os filhos permaneçam dentro do simplex e respeitem a cardinalidade (Quadro 21). A partir de um número aleatório $u \in [0, 1]$, calcula-se o parâmetro β , que controla o comportamento da recombinação: $\beta < 1$ gera filhos entre os valores dos pais; $\beta \approx 1$ produz filhos muito semelhantes aos pais; e $\beta > 1$ permite que os filhos extrapolem, ficando além dos pais em alguns ativos. Esse efeito é mais perceptível quando os pais diferem bastante entre si;

quando são parecidos, mesmo β elevado gera filhos próximos. Após o *crossover*, os filhos são ajustados para garantir pesos válidos e cardinalidade correta.

Quadro 21 – Implementação do *crossover* usando SBX

```
def _do(self, problem, X, **kwargs):
    _, n_matings, n_var = X.shape
    Y = np.full((self.n_offsprings, n_matings, n_var), np.nan)

    for k in range(n_matings):
        # Extrair pais
        p1 = X[0, k].copy()
        p2 = X[1, k].copy()

        # Garantir que pais somam 1 (segurança)
        p1 = p1 / p1.sum()
        p2 = p2 / p2.sum()

        # Simulated Binary Crossover (SBX)
        u = np.random.random()

        if u <= 0.5:
            beta = (2.0 * u) ** (1.0 / (self.eta + 1.0))
        else:
            beta = (1.0 / (2.0 * (1.0 - u))) ** (1.0 / (self.eta + 1.0))

        # Gerar filhos por interpolação convexa
        c1 = 0.5 * ((1 + beta) * p1 + (1 - beta) * p2)
        c2 = 0.5 * ((1 - beta) * p1 + (1 + beta) * p2)
```

Fonte: elaborado pelos autores.

O Quadro 22 ilustra um exemplo do funcionamento do operador, considerando dois portfólios pais válidos, cada um respeitando a soma unitária e a cardinalidade $K=4$. A partir deles, o operador aplica o mecanismo de recombinação do SBX com base no $\beta=1.11$, permitindo que os filhos extrapolem. Após a recombinação, os filhos são ajustados para garantir que permaneçam dentro do simplex (soma igual a 1, pesos não-negativos) e que mantenham exatamente quatro ativos com peso positivo.

Quadro 22 – Exemplo de um *crossover* de dois pais

Individuo	BBAS3	PETR4	VALE3	ITUB4	WEGE3
Pai 1	0.352	0.015	0.456	0.000	0.176
Pai 2	0.260	0.000	0.083	0.326	0.330
Filho 1	0.357	0.016	0.477	-0.018	0.167
Filho 2	0.255	-0.001	0.062	0.344	0.339
Filho 1 Normalizado	0,384	0.020	0.384	0.000	0.210
Filho 2 Normalizado	0.278	0.000	0.067	0.327	0.327

Fonte: elaborado pelo autor.

O operador `SimplexMutationCardConstraint` aplica, inicialmente, uma mutação baseada na transferência de peso entre ativos (Quadro 23). Essa estratégia é utilizada sempre que o portfólio possui menos que K ativos ou, caso já esteja no limite de cardinalidade, com probabilidade de 70%. O procedimento seleciona dois ativos com peso positivo e transfere parte do peso de um para o outro, preservando a soma total e evitando valores negativos. Por exemplo, considere um portfólio com os ativos $BBAS3 = 0,50$, $PETR4 = 0,30$ e $VALE3 = 0,20$. A mutação pode escolher $BBAS3$ e $VALE3$ e transferir uma pequena fração de $BBAS3$ para

VALE3, produzindo algo como $BBAS3 = 0,47$, $PETR4 = 0,30$ e $VALE3 = 0,23$. Esse ajuste local permite explorar novas combinações sem alterar a cardinalidade.

Quadro 23 – Transferência de peso entre ativos

```
# Mutação por transferência: move peso entre ativos
if n_active >= 2:
    # Escolhe dois ativos
    idx1, idx2 = np.random.choice(active_indices, 2, replace=False)
else:
    # Se menos de 2 ativos, escolhe qualquer par
    idx1, idx2 = np.random.choice(n_var, 2, replace=False)

# Calcular delta usando distribuição polynomial
u = np.random.random()

if u < 0.5:
    delta_q = (2.0 * u) ** (1.0 / (self.eta + 1.0)) - 1.0
else:
    delta_q = 1.0 - (2.0 * (1.0 - u)) ** (1.0 / (self.eta + 1.0))

# Magnitude da transferência (até 20% do menor valor)
magnitude = min(individual[idx1], individual[idx2]) * 0.2 * delta_q

# Transferir peso
individual[idx1] = individual[idx1] - magnitude
individual[idx2] = individual[idx2] + magnitude
```

Fonte: elaborado pelos autores.

A segunda estratégia de mutação é aplicada quando a cardinalidade já atingiu o limite de K ativos e ocorre com probabilidade de 30%. Nesse caso, o operador identifica o ativo com menor peso positivo e o substitui por um ativo atualmente fora do portfólio, transferindo totalmente o peso entre eles (Quadro 24). Por exemplo, suponha que o portfólio contenha os ativos $BBAS3 = 0,55$, $PETR4 = 0,35$ e $VALE3 = 0,10$, e que os ativos $ITUB4$ e $WEGE3$ estejam ausentes (peso zero). A mutação pode remover o ativo de menor peso ($VALE3$) e substituí-lo por $ITUB4$, resultando em $BBAS3 = 0,55$, $PETR4 = 0,35$, $VALE3 = 0,00$, $ITUB4 = 0,10$ e $WEGE3 = 0,00$. Esse mecanismo favorece a entrada de novos ativos sem alterar a cardinalidade total, promovendo diversidade estruturada na população.

Quadro 24 – Mutação por substituição: troca um ativo por um inativo

```
# Mutação por substituição: troca um ativo por um inativo
inactive_indices = np.where(~active_mask)[0]

if len(inactive_indices) > 0:
    # Escolhe ativo para remover (menor peso)
    remove_idx = active_indices[np.argmin(individual[active_indices])]

    # Escolhe ativo para adicionar (aleatório)
    add_idx = np.random.choice(inactive_indices)

    # Transfere todo o peso
    individual[add_idx] = individual[remove_idx]
    individual[remove_idx] = 0.0
```

Fonte: elaborado pelos autores.

3.1.6 Definição dos hiperparâmetros

A determinação dos hiperparâmetros ótimos para o algoritmo AGMO representa desafio fundamental para balancear qualidade das soluções e tempo computacional. O sistema

implementa metodologia sistemática de `grid search` combinada com análise de convergência para identificar configurações ideais para diferentes cenários de otimização.

O processo inicia-se testando combinações sistemáticas de tamanho populacional e número de gerações. Para cada configuração, o sistema executa múltiplas otimizações independentes e com quantidade de ativos diferentes, coletando métricas de qualidade como `hypervolume`, `spread` e `spacing` da fronteira de Pareto resultante. A repetição de execuções permite análise estatística robusta, quantificando variabilidade e identificando configurações consistentemente superiores.

Para algoritmos baseados em pontos de referência como o R-NSGA-II, a métrica tradicional de `hypervolume` apresenta limitações por não considerar as preferências do tomador de decisão durante a avaliação da qualidade. Por esta razão, o sistema implementa o R-Hypervolume, uma variação baseada no indicador R2 proposto por Hansen e Jaszkiwicz (1998), que mede a qualidade das soluções em relação aos pontos de referência definidos pelo perfil de risco do investidor.

O indicador R2 utiliza a Achievement Scalarizing Function (ASF) para quantificar a distância entre cada solução da fronteira de Pareto e os pontos de aspiração configurados. Matematicamente, o R2 é calculado como a média dos valores mínimos de ASF para cada ponto de referência (Quadro 25), onde valores menores indicam soluções mais próximas das preferências do investidor. Para manter consistência com outras métricas onde valores maiores representam melhor qualidade, o sistema aplica uma transformação `sigmoid` definida como $R\text{-}HV = 1/(1+R2)$, garantindo valores sempre positivos no intervalo $[0, 1]$, onde 1 representa qualidade perfeita.

Quadro 25 – Cálculo do indicador R2

```
# Calcula R2 indicator
r2_sum = 0.0
n_ref_points = len(normalized_ref_points)

for ref_point in normalized_ref_points:
    # Para cada ponto de referência, encontra a solução com menor ASF
    min_asf = float('inf')

    for solution in normalized_front:
        # ASF(a, z, w) = max_i {(a_i - z_i) / w_i}
        asf_values = np.where(weights > 1e-6,
                               (solution - ref_point) / weights,
                               (solution - ref_point) * 1e6)
        asf = np.max(asf_values)

        if asf < min_asf:
            min_asf = asf

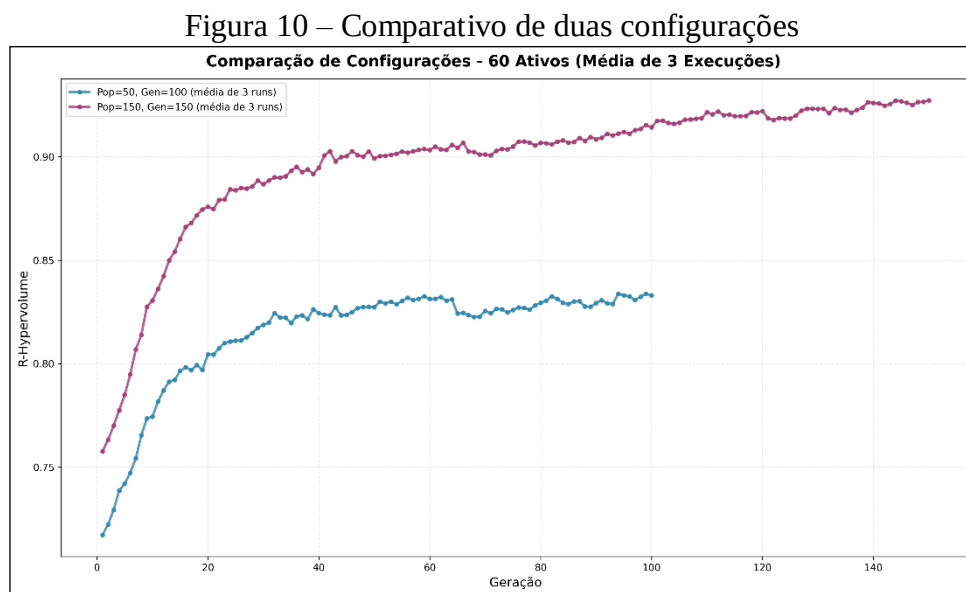
    r2_sum += min_asf

# R2 = média dos mínimos ASF
r2 = r2_sum / n_ref_points
```

Fonte: elaborado pelos autores.

A normalização dos objetivos para o cálculo do R-Hypervolume requer atenção especial para garantir comparabilidade entre gerações distintas. O sistema adota estratégia de fazer uma execução prévia com hiperparâmetros ótimos para obter o ponto ideal, composto pelos menores valores observados em cada objetivo, e o ponto nadir, formado pelos maiores valores registrados. Essa escala permite transformar todos os objetivos para a escala $[0, 1]$, facilitando sua comparação e garantindo que nenhum objetivo domine os demais devido a diferenças de magnitude.

O módulo `ConvergenceTracker` registra automaticamente as métricas de qualidade a cada geração, permitindo análise detalhada da evolução do algoritmo. As visualizações geradas incluem a curva de convergência do R-Hypervolume, o tamanho da fronteira de Pareto, e métricas de diversidade como `spread` e `spacing`. Estas informações são fundamentais para validar a adequação dos hiperparâmetros selecionados e identificar o ponto de convergência onde incrementos adicionais em gerações resultam em ganhos marginais decrescentes. A Figura 10 representa um exemplo dessa análise, com uma comparação entre uma média de 3 execuções com população inicial de 50 indivíduos e 100 gerações e outra com população inicial de 150 indivíduos e 150 gerações.



Fonte: elaborado pelos autores.

A análise gráfica do processo de *tuning* revela como o indicador R-Hypervolume evolui ao longo das execuções, permitindo avaliar a estabilidade e a qualidade das soluções geradas pelo algoritmo. Observa-se que os valores de hipervolume começam mais dispersos nas primeiras execuções, indicando variação maior na qualidade das fronteiras produzidas. À medida que o processo avança, os pontos convergem para uma faixa mais estreita e consistente, demonstrando que o conjunto de hiperparâmetros testados conduz o algoritmo a soluções mais

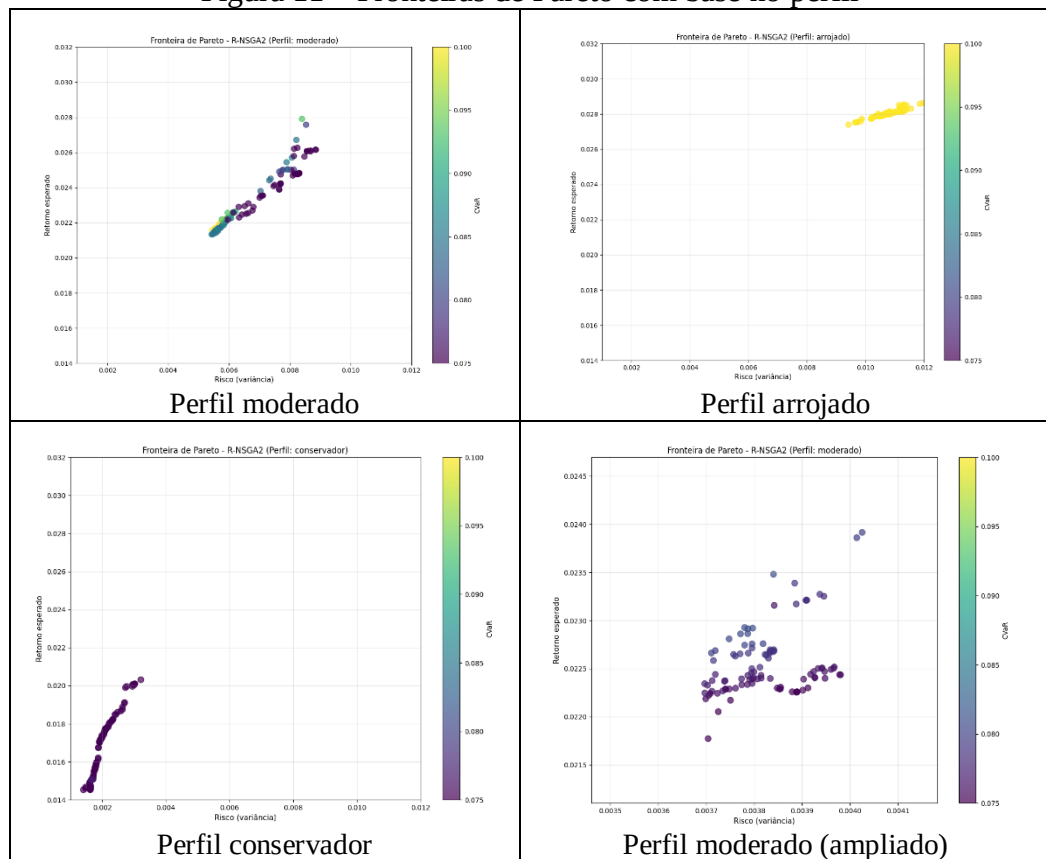
estáveis e com melhor equilíbrio entre os objetivos. Nesse exemplo nota-se que a configuração com população 150 e 150 gerações apresenta R-Hypervolume consistentemente superior, o que pode ser atribuído ao maior tamanho populacional, capaz de explorar com mais amplitude o espaço de busca e produzir fronteiras mais completas e diversificadas.

Os resultados do *tuning* são persistidos no banco de dados, criando biblioteca de configurações ótimas indexadas por número de ativos e perfil de risco. Durante execuções subsequentes, o sistema consulta automaticamente esta biblioteca, aplicando hiperparâmetros previamente otimizados para cenários similares.

3.1.6.1 Resultados e discussões

A Figura 11 apresenta as fronteiras eficientes obtidas na última geração do algoritmo R-NSGA-II para diferentes perfis de risco (conservador, moderado e arrojado). Cada ponto nos gráficos representa um indivíduo da população final, ou seja, uma carteira de investimentos viável, caracterizada pelos valores de risco (variância), retorno esperado e CVaR.

Figura 11 – Fronteiras de Pareto com base no perfil



Fonte: elaborado pelos autores.

As fronteiras de Pareto dos perfis conservador, moderado e arrojado mostram com clareza o *trade-off* entre retorno e risco. O perfil arrojado concentra soluções com retornos mais altos, porém acompanhadas de variância e CVaR também elevados, enquanto o perfil

conservador reúne alternativas mais estáveis, mas com retornos menores. Observando a fronteira do perfil arrojado, por exemplo, é possível ver um conjunto de soluções com retorno acima de 0,028, mas com os níveis de variância e CVaR nas máximas das escalas representadas. Essa leitura visual ajuda a compreender como o algoritmo distribui as soluções conforme o nível de risco escolhido pelo usuário. Para complementar a análise, recomenda-se apresentar métricas como retorno anualizado, volatilidade e CVaR para cada perfil, permitindo quantificar de forma objetiva as diferenças observadas.

Embora o R-NSGA-II gere uma fronteira de Pareto com múltiplas soluções não dominadas, o sistema seleciona automaticamente aquela mais alinhada ao perfil de risco especificado. O procedimento inicia pela normalização dos objetivos, utilizando os valores mínimos e máximos observados na própria fronteira de Pareto calculada pelo algoritmo, assim tendo um *range* de valores para a normalização. Essa abordagem garante que todos os objetivos, independentemente de suas unidades ou escalas originais, sejam transformados para um intervalo comum [0, 1] (Quadro 26).

Quadro 26 – Escolha da melhor carteira das soluções

```
def _choose_best_portfolio(self, objectives, solutions):
    # Usa configuração centralizada (constantes do módulo)
    ref_point = REFERENCE_POINTS_CONFIG[self.risk_level][0] # [0] extrai o primeiro
    array da matriz
    weights = WEIGHTS_CONFIG[self.risk_level]

    # Normaliza os objetivos para [0, 1]
    objectives_normalized = objectives.copy()
    for i in range(objectives.shape[1]):
        col = objectives[:, i]
        min_val, max_val = col.min(), col.max()

        if max_val - min_val > 1e-10:
            objectives_normalized[:, i] = (col - min_val) / (max_val - min_val)
        else:
            objectives_normalized[:, i] = 0.0

    # Calcula ASF para cada solução
    asf_values = []
    for obj in objectives_normalized:
        # ASF = max_i { (obj[i] - ref[i]) / weight[i] }
        # Menor ASF = mais próximo do reference point
        asf_components = (obj - ref_point) / weights
        asf = np.max(asf_components)
        asf_values.append(asf)

    asf_values = np.array(asf_values)

    # Seleciona solução com menor ASF (mais próxima do reference point)
    best_idx = np.argmin(asf_values)

    return solutions[best_idx]
```

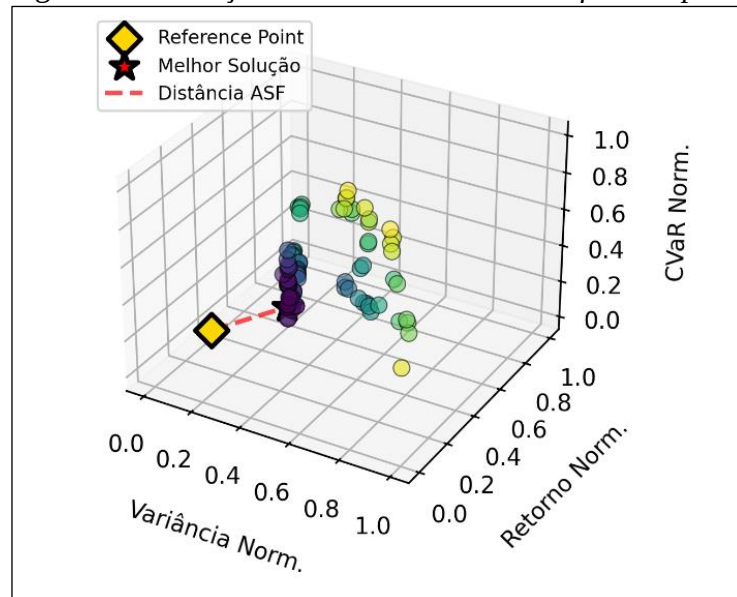
Fonte: elaborado pelos autores.

Em seguida, aplica-se a ASF sobre as soluções normalizadas, considerando os pontos de referência estabelecidos para cada perfil de risco. O ASF mede o quão próxima cada solução está desses *reference points*. Assim, a recomendação final corresponde à solução que apresenta

o menor valor de ASF, pois ela representa a alternativa mais coerente com as preferências associadas ao perfil selecionado.

Na Figura 12 são apresentadas as soluções normalizadas no intervalo $[0, 1]$ obtidas em uma execução com perfil conservador; os eixos correspondem, respectivamente, à variância normalizada, ao retorno normalizado e ao CVaR normalizado. O losango amarelo indica o *reference point* definido para esse perfil $[0.3, 0.05, 0.05]$, ou seja, preferências que priorizam baixo risco e baixa perda de cauda, e a estrela marca a melhor solução selecionada, a mais próxima desse ponto, com valores $[0.49, 0.25, 0.128]$. Esse posicionamento evidencia que a carteira escolhida equilibra um retorno compatível com o gosto conservador, mantendo variância e CVaR reduzidos em relação às demais soluções da fronteira.

Figura 12 – Soluções normalizadas com o *reference point*



Fonte: elaborado pelos autores.

3.1.6.2 Backtest e comparação com Benchmark

O serviço também oferece a possibilidade de definir uma data de referência como parâmetro de entrada na otimização. Quando essa data é informada, o método limita o conjunto de dados históricos apenas ao período anterior a ela, criando uma simulação realista de como a carteira teria sido construída com as informações disponíveis naquele momento. Esse recurso funciona como uma forma de *backtest* interno, permitindo avaliar se a carteira otimizada com dados realmente passados teria apresentado um comportamento consistente ao longo do tempo.

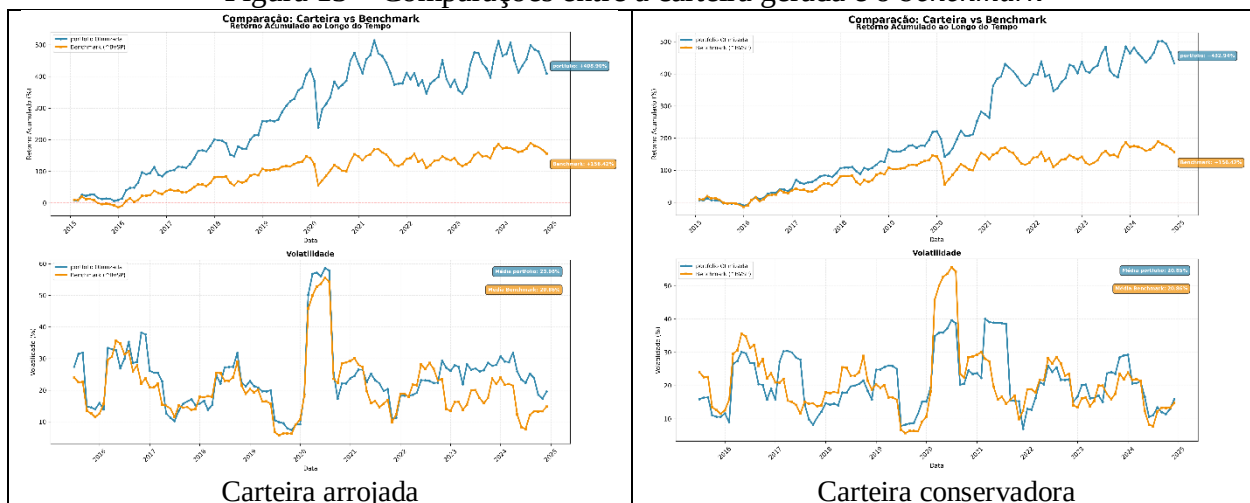
A partir dessa funcionalidade, implementou-se uma rotina específica para comparar o desempenho da carteira consolidada no *backtest* com um *benchmark* de mercado. No caso deste

projeto, utilizou-se o Ibovespa como referência, por ser o principal índice da bolsa brasileira e um parâmetro amplamente adotado na avaliação de estratégias de investimento.

Para demonstrar a aplicabilidade do sistema em diferentes contextos de investimento, realizaram-se *backtests* de duas carteiras com perfis de risco opostos: uma conservadora e outra arrojada, ambas otimizadas para um horizonte de 10 anos sem restrição de ativos específicos e limite de 10 ativos por carteira.

Conforme demonstrado na Figura 13, ambas carteiras superaram o índice Ibovespa ao longo do período analisado, porém com comportamentos distintos. A carteira arrojada obteve maior volatilidade e quedas mais acentuadas durante períodos de crise. A carteira conservadora, apresentou trajetória mais estável e maior resiliência em momentos de alta volatilidade, e nesse exemplo até conseguiu superar o retorno acumulado da carteira arrojada.

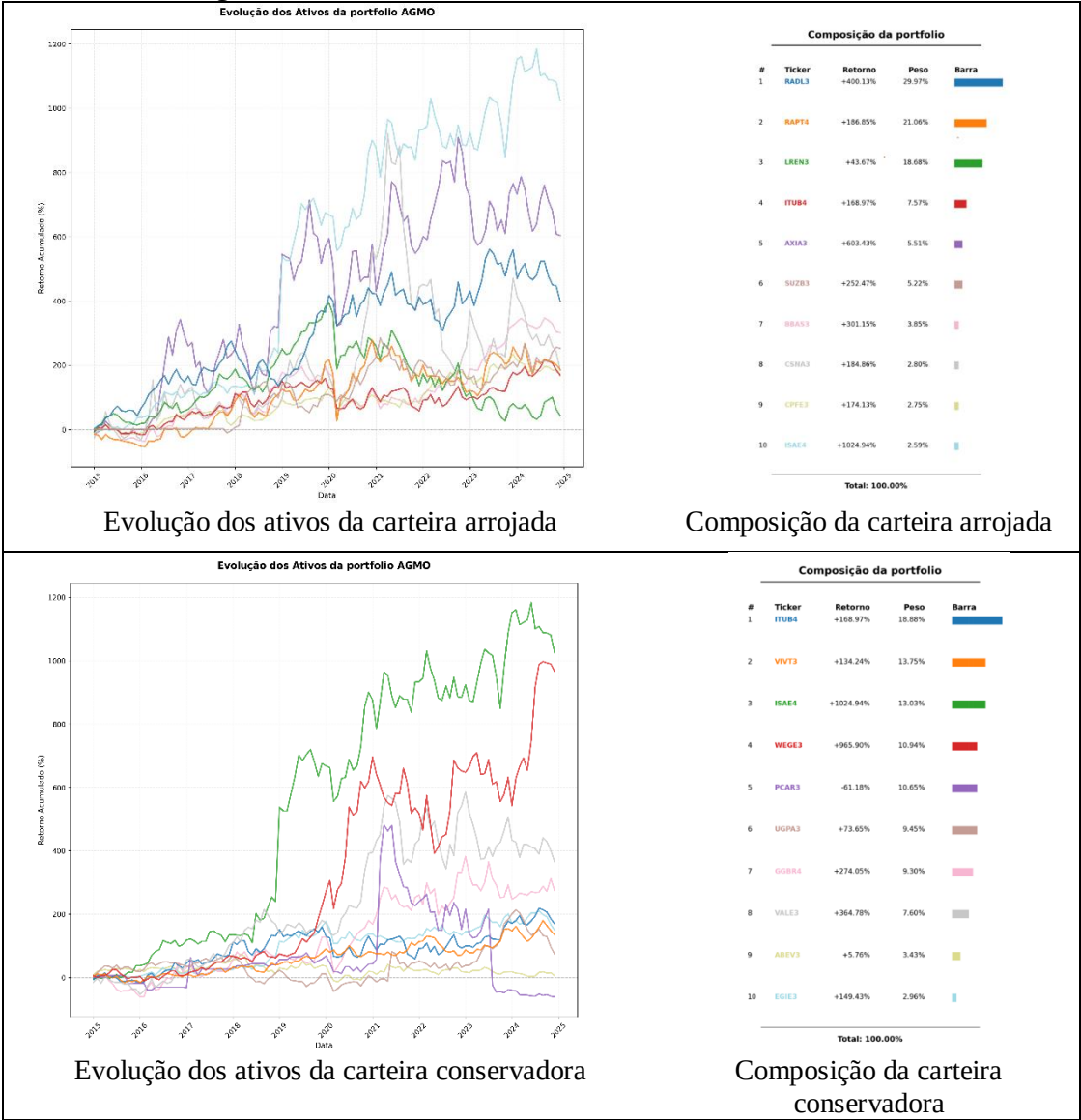
Figura 13 – Comparações entre a carteira gerada e o *benchmark*



Fonte: elaborado pelos autores.

A Figura 14 detalha a composição e contribuição individual dos ativos de cada carteira, comprovando o retorno acumulado do portfólio. A heterogeneidade dos ativos confirma que a otimização multiobjetivo seleciona não apenas ativos individualmente promissores, mas aqueles que, em conjunto, oferecem diversificação eficiente.

Figura 14 – Retornos individuais dos ativos das carteiras



Fonte: elaborado pelos autores.

Os gráficos comportamentais da Figura 14 evidenciam como o algoritmo ajustou a alocação conforme o perfil de risco. Para o perfil arrojado, o algoritmo concentrou grande parte do portfólio em ativos com histórico de forte valorização e maior volatilidade, pois estes maximizaram o retorno esperado dentro da fronteira de Pareto. Já a carteira conservadora apresenta distribuição mais equilibrada e foco em ativos com trajetória de crescimento mais estável e de setores perenes, como de bancos e elétricas, escolhidos pelo algoritmo por reduzirem o risco extremo e suavizarem as oscilações de retorno ao longo do período.

3.1.7 API de acesso ao modelo AGMO

A arquitetura da API REST segue princípios RESTful estabelecidos, implementando endpoints semanticamente corretos para operações CRUD sobre os recursos do sistema. A

estrutura modular baseada em Flask Blueprints organiza as rotas em módulos coesos, facilitando a manutenção e a evolução independente das funcionalidades (Quadro 27). Além disso, os *endpoints* seguem um padrão unificado, com prefixos que identificam claramente cada módulo da aplicação, permitindo que o *frontend* navegue e consuma os serviços de forma consistente; por exemplo, a autenticação é exposta na rota `api/auth/login`. O serviço é disponibilizado para a aplicação React por meio da habilitação do Cross-origin Resource Sharing (CORS), garantindo que o domínio do *frontend* tenha permissão para acessar a API de maneira segura. Também é implementado um mecanismo de *rate limiting* via Flask-Limiter, que previne ataques de força bruta, aplicando limites diferenciados para *endpoints* sensíveis, como o de autenticação (10 tentativas por minuto).

Quadro 27 – Requisição de login da API

```
auth_bp = Blueprint('auth', __name__)
logger = logging.getLogger(__name__)

@auth_bp.route('/login', methods=['POST'])
@limiter.limit("10 per minute")
def login():
    data = request.get_json()

    email = data.get('email') if data else None
    password = data.get('senha') if data else None

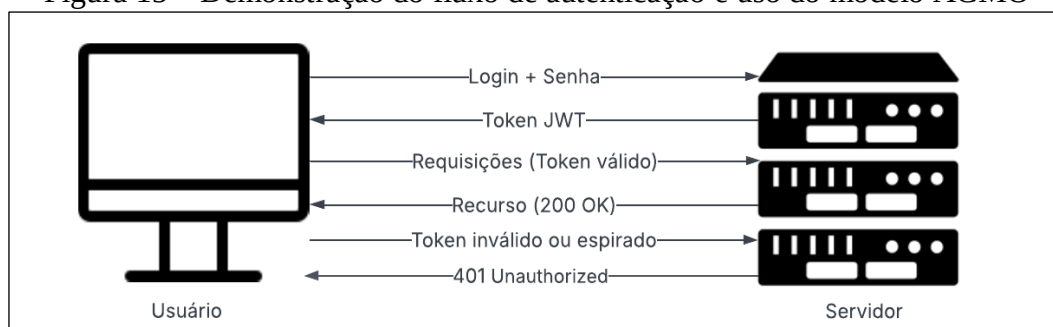
    token, result = AuthService.login(email, password)

    if token:
        return jsonify({
            'token': token,
            'usuario': result
        }), 200
    else:
        return jsonify({'erro': result}), 401
```

Fonte: elaborado pelos autores.

O sistema de autenticação da API implementa JSON Web Tokens (JWT), escolha fundamentada em suas características de *statelessness* e capacidade de carregar *claims* customizados. Cada *token* contém identificador único do usuário e *timestamp* de expiração, eliminando necessidade de armazenamento *server-side* de sessões. Na Figura 15 é possível visualizar o fluxo de autenticação com o *token*.

Figura 15 – Demonstração do fluxo de autenticação e uso do modelo AGMO



Fonte: elaborado pelos autores.

O *middleware* de proteção intercepta requisições a *endpoints* protegidos, extraindo e validando *tokens* do *header Authorization*. A validação verifica assinatura criptográfica, expiração temporal e revogação opcional através de *blacklist*. Requisições com *tokens* inválidos ou ausentes recebem resposta 401 *Unauthorized* imediata.

3.2 PLATAFORMA WEB (FRONTEND)

Nesta seção apresenta-se a plataforma web desenvolvida, o projeto foi desenvolvido utilizando React 18 aliado ao Vite, seguindo a arquitetura de aplicação de página única (Single Page Application – SPA). A interface adota Tailwind CSS para estilização baseada em classes utilitárias, permitindo a construção eficiente de *layouts* responsivos. A comunicação com o *backend* ocorre por meio de uma API REST, intermediada pela biblioteca Axios, cuja instância centralizada localiza-se em `src/services/api.js`. Esta instância é responsável por inserir o *token* JWT no cabeçalho *Authorization* e por aplicar tratamento global de erros, garantindo consistência em todas as requisições.

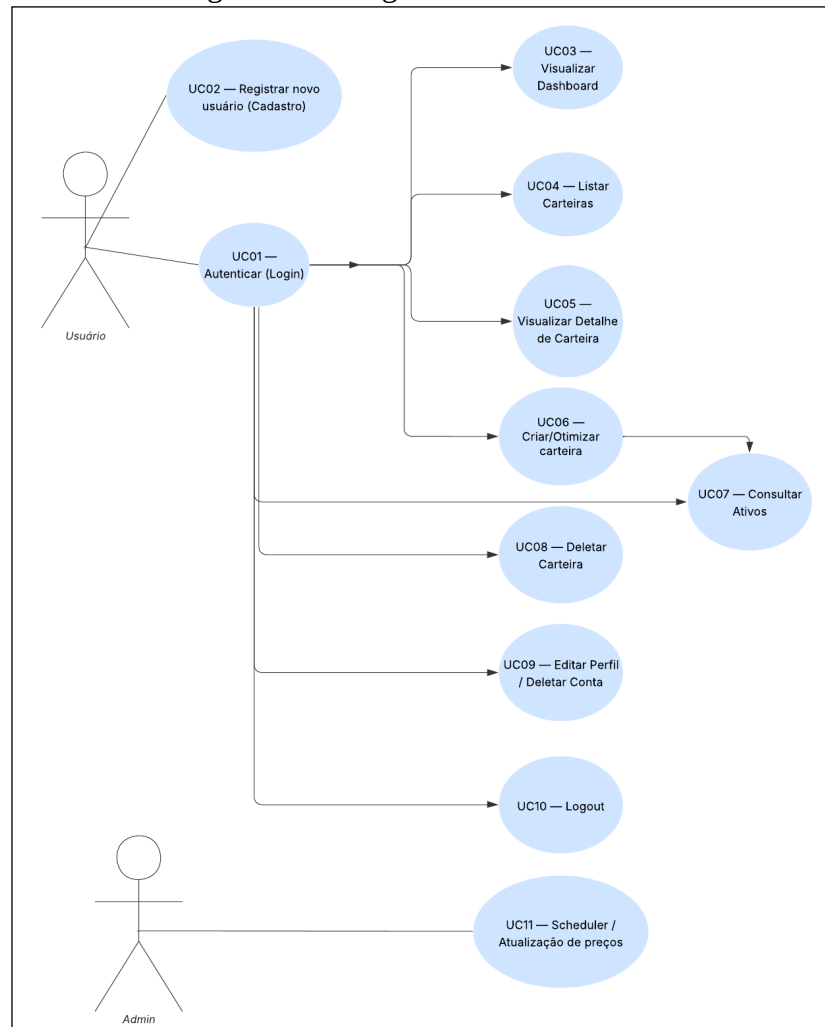
A arquitetura do *frontend* segue princípios de modularidade e separação de responsabilidades. A estrutura de pastas organiza o código em camadas claras: roteamento (`App.jsx` e `RequireAuth.jsx`), *layout* (`Layout.jsx`), páginas (*pages*), componentes reutilizáveis (*components*), serviços (`api.js`) e ativos (*assets*). O roteamento é gerido por React Router v6, com rotas públicas e protegidas; o componente `RequireAuth.jsx` atua como guarda, verificando a presença do JWT no *localStorage*. A comunicação com o servidor é centralizada em uma instância de Axios em `api.js`, que configura a *baseURL* a partir da variável de ambiente `VITE_API_URL` e implementa *interceptors* para injeção automática do cabeçalho de autorização e tratamento padronizado de respostas e erros.

Os fluxos de interação seguem um padrão unidirecional: ações de interface disparam requisições ao *backend*, que retornam dados usados para atualizar o estado e renderizam a UI. Exemplo representativo: ao submeter o formulário de otimização, `OtimizarForm` valida entradas no cliente (nome, capital, quantidade mínima de ativos), monta `restricoes_ativos` e envia o *payload*; após retorno bem-sucedido a lista de carteiras é atualizada e a nova carteira é selecionada automaticamente. Erros de rede e respostas do servidor são capturados e exibidos ao usuário; o interceptor central trata especificamente respostas 401 para forçar nova autenticação quando necessário. A partir deste contexto, nas próximas subseções detalha-se a especificação, implementação e testes da plataforma web desenvolvida para apoiar investidores de diferentes níveis de experiência na tomada de decisões

3.2.1 Diagrama de Casos de Uso

A Figura 16 apresenta o diagrama de casos de uso da aplicação web, que representa graficamente as interações entre os usuários e as funcionalidades do sistema, evidenciando os principais fluxos de operação e responsabilidades de cada ator.

Figura 16 – Diagrama de casos de uso



Fonte: elaborado pelos autores.

Conforme ilustrado na Figura 16, o ator *Usuário* inicia sua interação com o sistema realizando o processo de autenticação através do caso de uso UC01 – Autenticar (Login). Caso ainda não possua cadastro, o usuário pode registrar-se na plataforma utilizando o UC02 – Registrar novo usuário, fornecendo suas informações pessoais e de contato. Após o login bem-sucedido, o usuário tem acesso ao conjunto de funcionalidades disponíveis no *Dashboard* principal, onde pode visualizar informações gerais sobre suas carteiras e desempenho financeiro (UC03).

A partir do *Dashboard*, o usuário pode navegar para a lista completa de carteiras cadastradas em sua conta (UC04), onde encontra todas as carteiras financeiras que gerencia. Ao

selecionar uma carteira específica, o sistema apresenta detalhes aprofundados, incluindo saldo, movimentações e distribuição de ativos (UC05). Caso deseje criar uma carteira ou otimizar carteiras existentes, o usuário utiliza o UC06 – Criar/Otimizar carteira, no qual pode configurar preferências, objetivos financeiros e estratégias de investimento.

Para complementar sua análise, o usuário pode consultar informações de seus investimentos por meio do UC07 – Consultar ativos, visualizando dados atualizados sobre ações, criptomoedas ou outros ativos disponíveis em sua carteira. Quando necessário, o usuário pode excluir uma carteira indesejada (UC08), procedimento que exige confirmação para evitar exclusões indevidas. Além disso, o usuário pode gerenciar seus próprios dados por meio do UC09 – Editar Perfil / Deletar Conta, onde é possível alterar informações cadastradas ou solicitar a remoção definitiva da conta.

O usuário também pode efetuar o UC10 – Logout, permitindo encerrar sua sessão com segurança sempre que necessário. Já o ator Admin possui permissões adicionais e desempenha funções específicas relacionadas ao funcionamento interno da plataforma. Sua principal responsabilidade está representada no UC11 – Scheduler / Atualização de preços, que envolve o controle e manutenção das rotinas automáticas do sistema, garantindo que as cotações de ativos e demais dados financeiros sejam atualizados adequadamente em intervalos definidos.

3.2.2 Requisitos Funcionais e Não Funcionais

Esta seção descreve o levantamento dos requisitos, onde são apresentados os Requisitos Funcionais e Não Funcionais da plataforma web responsável pela interação entre o usuário e o AGMO.

- a) autenticação de usuário: a aplicação deve permitir autenticação via e-mail e senha, fornecendo *token* válido e redirecionando ao *dashboard* (RF);
- b) registro de novo usuário: a aplicação deve disponibilizar formulário de cadastro, permitindo criação e posterior login (RF);
- c) visualizar *dashboard* com prévias: a aplicação deve exibir prévias de carteiras, métricas e demais dados resumidos (RF);
- d) listar carteiras do usuário: a aplicação deve apresentar todas as carteiras cadastradas, permitindo seleção e exclusão (RF);
- e) visualizar detalhe e composição de carteira: a aplicação deve apresentar parâmetros e composição da carteira, incluindo tabela e gráfico (RF);
- f) criar / otimizar carteira: a aplicação deve possibilitar envio de parâmetros

financeiros e restrições para criação de carteira otimizada (RF);

- g) consultar ativos: a aplicação deve permitir listagem e filtragem de ativos para composição de carteiras (RF);
- h) deletar carteira: a aplicação deve permitir exclusão de carteiras mediante confirmação (RF);
- i) editar perfil / deletar conta: a aplicação deve permitir edição de dados do usuário e exclusão de conta (RF);
- j) navegação protegida por sessão: a aplicação deve restringir acesso às rotas internas mediante verificação de *token* (RF);
- k) segurança: o *token* deve ser armazenado e enviado em autorização *Bearer*. Em produção, recomenda-se uso de cookie `httpOnly` (RNF);
- l) performance: páginas essenciais devem carregar em menos de 2 segundos em ambiente local (RNF);
- m) usabilidade e acessibilidade: a interface deve ser responsiva, acessível e com foco adequado em elementos interativos (RNF);
- n) manutenibilidade: a estrutura deve manter organização modular em `components/`, `pages/` e `services/` (RNF);
- o) compatibilidade: a aplicação deve suportar navegadores modernos com `ES Modules` (RNF);
- p) observabilidade: mensagens de erro devem ser apresentadas ao usuário, e falhas registradas no *console* (RNF).

A seguir, são apresentadas as tecnologias e ferramentas empregadas no desenvolvimento da plataforma web, responsáveis pela construção da interface, comunicação com a API, autenticação, armazenamento de dados e execução da lógica de negócio.

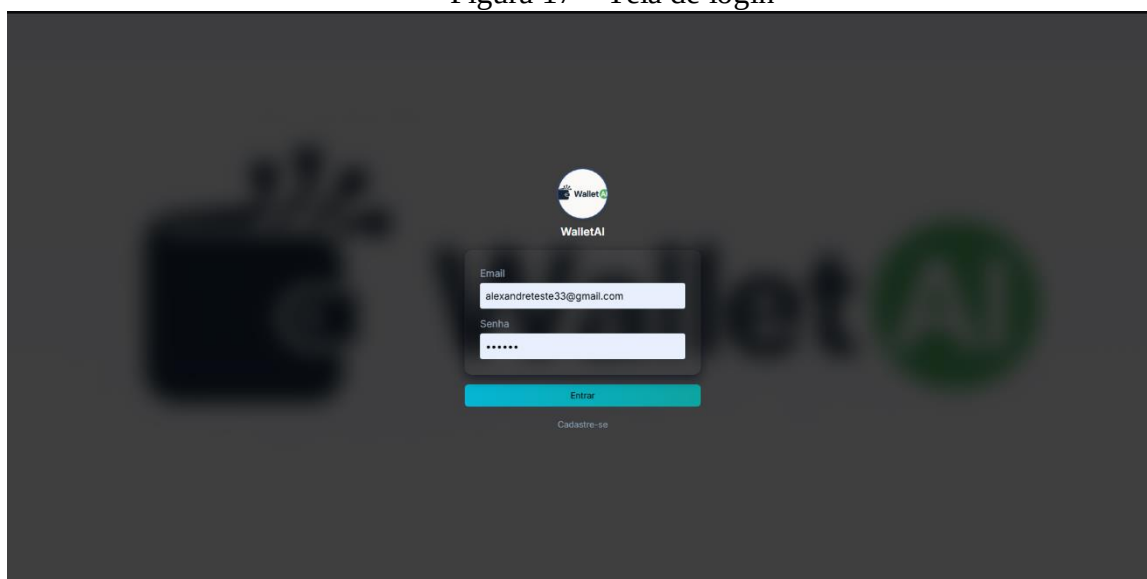
- a) React: biblioteca JavaScript para construção de interfaces de usuário baseada em componentes reutilizáveis;
- b) Tailwind CSS: *framework* CSS utilitário que permite estilização rápida sem a necessidade de arquivos CSS extensos;
- c) Vite: ferramenta moderna de *bundling* para projetos JavaScript, com inicialização rápida e *hot reload* eficiente;
- d) Axios: biblioteca para consumo de APIs REST via HTTP;
- e) JWT: padrão aberto para criação de *tokens* utilizados em autenticação;
- f) localStorage: Mecanismo de armazenamento persistente no navegador.

3.2.3 Operacionalidade da aplicação web

Esta seção apresenta as principais telas da aplicação web WalletAI, destacando sua finalidade, funcionalidades e relação direta com os requisitos definidos na etapa de análise. Cada interface foi projetada para garantir usabilidade, organização da informação e fluxo intuitivo para o usuário final.

A Figura 17 (tela de *Login*) é a porta de entrada para a aplicação, onde o usuário insere suas credenciais, como email e senha, para autenticar-se. Ao submeter o formulário, a aplicação valida as informações com o *backend* e, se as credenciais forem corretas, o usuário é redirecionado para o *Dashboard*, com o *token* de autenticação salvo no *localStorage* para garantir o acesso contínuo às funcionalidades protegidas. Caso as credenciais sejam inválidas ou ocorra algum erro de rede, o usuário será informado por meio de mensagens de erro na própria tela.

Figura 17 – Tela de login



Fonte: elaborado pelos autores.

O Quadro 28 mostra o processo de envio das credenciais ao servidor, armazenamento do *token* JWT recebido em *localStorage* e o redirecionamento do usuário para o *Dashboard*. Esse fluxo também trata e exibe mensagens de erro quando a autenticação falha.

Quadro 28 – Exemplo do *handler* de login em Login.jsx

```
import api from '../services/api.js';
import { useNavigate } from 'react-router-dom';

async function handleLogin(values, setError, setLoading) {
  try {
    setLoading(true);
    const res = await api.post('/api/login', { email: values.email, senha: values.senha });
    const { token, usuario } = res.data;
    localStorage.setItem('token', token);
    localStorage.setItem('usuario', JSON.stringify(usuario));
    navigate('/dashboard');
  } catch (err) {
    setError(err.response?.data?.erro || err.message);
  } finally {
    setLoading(false);
  }
}
}
```

Fonte: elaborado pelos autores.

A Figura 18 (tela de Cadastro de Usuário) é voltada para usuários que ainda não possuem uma conta na aplicação. Nela, o usuário preenche um formulário com suas informações pessoais e credenciais para criação da conta. Após o cadastro bem-sucedido, a aplicação redireciona automaticamente o usuário para a tela de *Login* ou, se registrado no banco de dados, faz a autenticação automaticamente, permitindo que ele comece a usar o sistema imediatamente.

Figura 18 – Tela de cadastro de usuário

Fonte: elaborado pelos autores.

O Quadro mostra o processo de envio dos dados preenchidos no formulário de cadastro ao *endpoint* responsável pela criação de usuários (POST /api/usuarios), retornando mensagem de sucesso ou falha.

Quadro 29 – Exemplo de submissão em Register.jsx

```
import api from '../services/api.js';

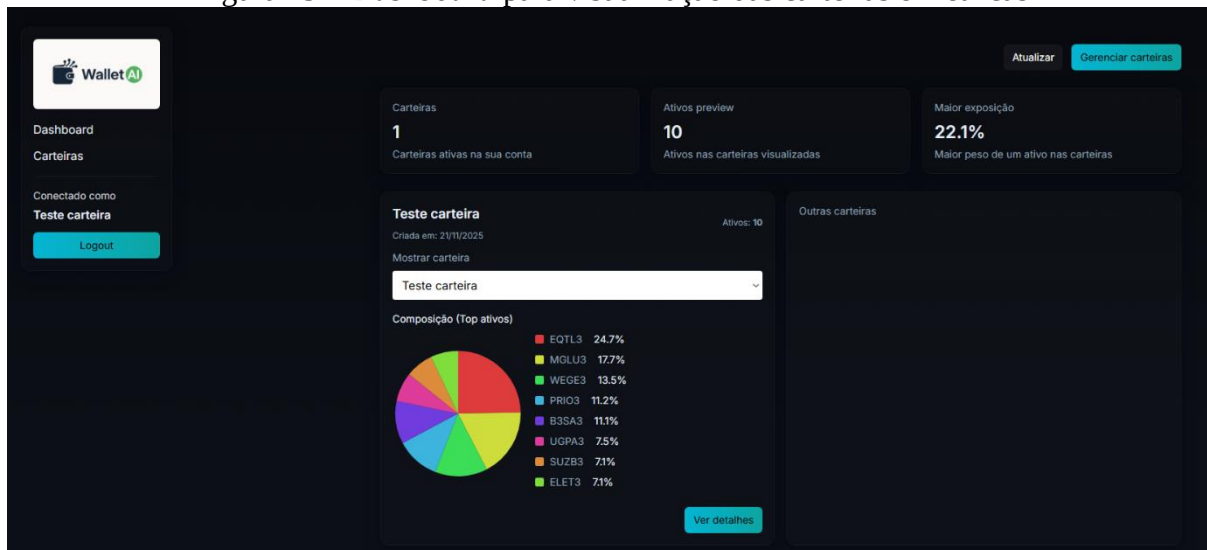
async function handleRegister(formData, setError, setSuccess) {
  try {
    const res = await api.post('/api/usuarios', formData);
    setSuccess('Cadastro realizado com sucesso. Você pode fazer login agora.');
```

// opcional: autenticar automaticamente

```
// await autoLogin(formData.email, formData.senha);
  } catch (err) {
    setError(err.response?.data?.erro || err.message);
  }
}
```

Fonte: elaborado pelos autores.

A Figura 19 (tela de *Dashboard*) oferece uma visão geral do usuário sobre suas carteiras e métricas relacionadas. Ao acessar essa tela, busca-se os dados resumidos sobre as carteiras do usuário e as exibe em forma de cards, apresentando informações como saldo, performance e composição. O usuário também tem atalhos para as principais funcionalidades, como criar carteiras ou visualizar detalhes das existentes, facilitando a navegação no sistema.

Figura 19 – *Dashboard* para visualização das carteiras e métricas

Fonte: elaborado pelos autores.

O Quadro 30 demonstra como é realizada a busca da carteira do usuário (`GET /api/carteiras/summary`), na montagem do componente para popular os cards do *Dashboard* com métricas como saldo, performance e composição.

Quadro 30 – Busca de resumo no Dashboard.jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api.js';

function Dashboard() {
  const [summary, setSummary] = useState(null);

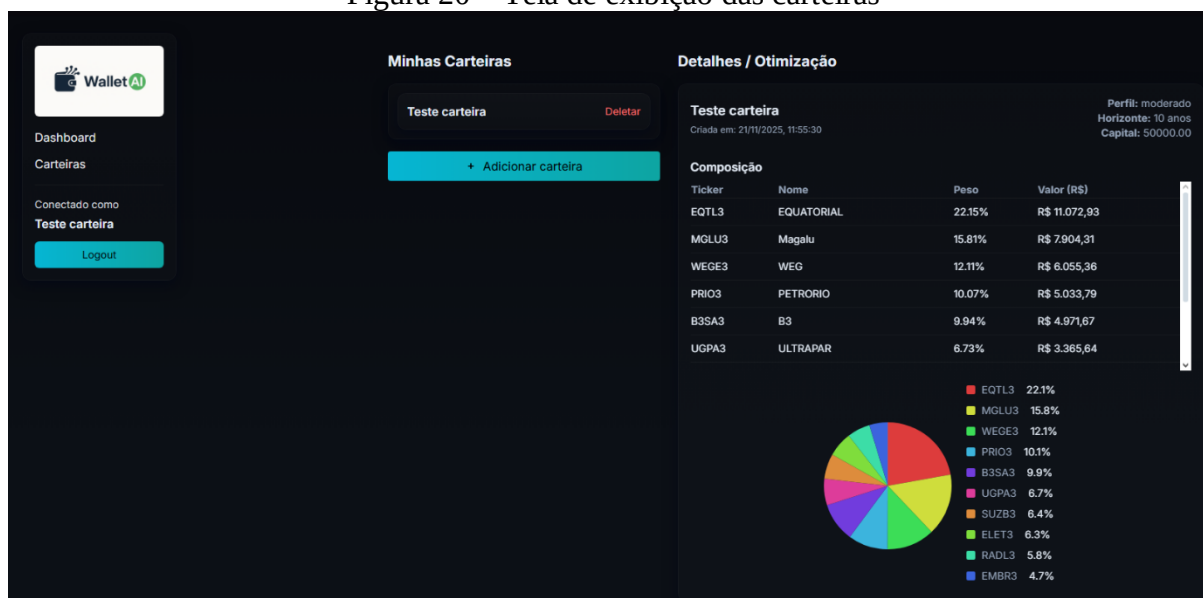
  useEffect(() => {
    api.get('/api/carteiras/summary')
      .then(res => setSummary(res.data))
      .catch(err => console.error(err));
  }, []);

  // renderizar cards com summary...
}
```

Fonte: elaborado pelos autores.

A Figura 20 (tela de Carteiras) exibe a lista de todas as carteiras cadastradas pelo usuário. Cada carteira é listada de maneira organizada, permitindo que o usuário a selecione para visualizar mais detalhes ou realizar ações como editar ou excluir. A partir desta tela, o usuário pode também iniciar o processo de criação de uma nova carteira, utilizando o formulário de otimização disponível na aplicação.

Figura 20 – Tela de exibição das carteiras



Fonte: elaborado pelos autores.

O Quadro 31 mostra como é feita a requisição da lista completa de carteiras (GET /api/carteiras), armazenamento do estado local e a disponibilização das funções para seleção e atualização da listagem após operações de criação ou exclusão.

Quadro 31 – Listagem e seleção em Carteiras.jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api.js';

function Carteiras() {
  const [carteiras, setCarteiras] = useState([]);

  useEffect(() => {
    fetchCarteiras();
  }, []);

  function fetchCarteiras() {
    api.get('/api/carteiras')
      .then(res => setCarteiras(res.data))
      .catch(err => console.error(err));
  }

  function handleSelect(id) {
    // seleciona a carteira e exibe detalhes
    setSelectedId(id);
  }

  // render lista com carteiras.map(...)
}
```

Fonte: elaborado pelos autores.

A Figura 21 (tela de Detalhes de Carteira) mostra informações detalhadas sobre a carteira selecionada, incluindo a composição dos ativos, perfil de risco, horizonte de investimento e capital investido. A tela apresenta esses dados em formato de tabela e gráficos, como um gráfico de pizza (*PieChart*), para uma visualização clara da distribuição dos ativos. O usuário também pode ver os parâmetros utilizados na criação da carteira e entender melhor como as decisões de otimização foram aplicadas.

Figura 21 – Tela de criação de carteiras

Fonte: elaborado pelos autores.

No Quadro 32 realiza-se o carregamento dos dados de uma carteira selecionada (`GET /api/carteiras/:id`), retornando a composição para um componente gráfico (*PieChart*) para representar visualmente a distribuição dos ativos.

Quadro 32 – Carregar detalhe utilizando PieChart em CarteiraDetail.jsx

```
import { useEffect, useState } from 'react';
import api from '../services/api.js';
import PieChart from '../components/PieChart.jsx';

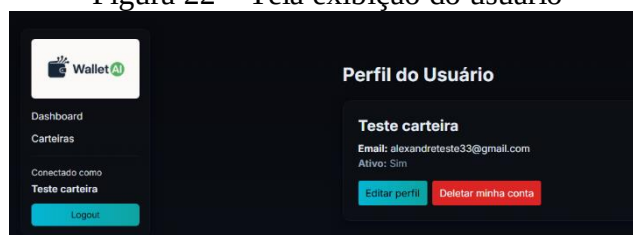
function CarteiraDetail({ id }) {
  const [carteira, setCarteira] = useState(null);
  useEffect(() => {
    api.get(`/api/carteiras/${id}`)
      .then(res => setCarteira(res.data))
      .catch(err => console.error(err));
  }, [id]);

  if (!carteira) return <div>Carregando...</div>;
  return (
    <>
      <PieChart items={carteira.composicao} />
      { /* tabela de composição e parâmetros */ }
    </>
  );
}
```

Fonte: elaborado pelos autores.

A Figura 22 (tela de Perfil) permite que o usuário visualize e edite suas informações pessoais, como nome, email, e preferências de conta. Caso o usuário queira excluir sua conta, essa opção também está disponível nesta tela, onde ele pode solicitar a remoção de todos os dados associados. Após realizar alterações, o usuário pode salvar as modificações diretamente no sistema.

Figura 22 – Tela exibição do usuário



Fonte: elaborado pelos autores.

O Quadro 33 apresenta o trecho de código que realiza as alterações de perfil do usuário via PUT /api/perfil ou a remoção da conta com DELETE /api/usuarios/me, garantindo também a limpeza das credenciais locais quando a exclusão é confirmada.

Quadro 33 – Salvar alterações e deletar conta em Profile.jsx

```
import api from '../services/api.js';
async function saveProfile(updated, setMessage) {
  try {
    await api.put('/api/perfil', updated);
    setMessage('Perfil atualizado com sucesso.');
```

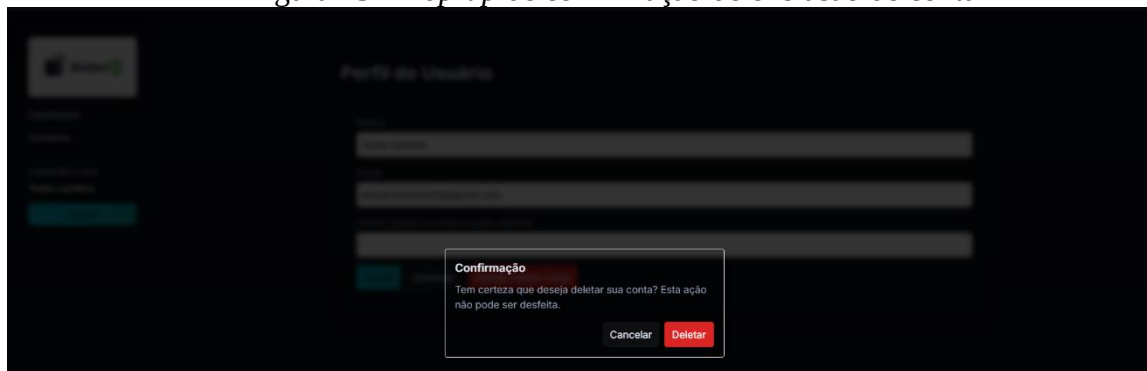
```
  } catch (err) {
    setMessage(err.response?.data?.erro || err.message);
  }
}

async function deleteAccount(setMessage) {
  try {
    await api.delete('/api/usuarios/me');
    localStorage.removeItem('token');
    // redirecionar para login...
  } catch (err) {
    setMessage(err.response?.data?.erro || err.message);
  }
}
```

Fonte: elaborado pelos autores.

A Figura 23 (tela de *Logout*) é simples e direta: ao clicar no botão de *logout*, o usuário finaliza sua sessão, o *token* de autenticação é removido do `localStorage` e ele é redirecionado para a tela de *Login*. Essa tela garante a segurança e a privacidade do usuário, permitindo o encerramento seguro da sessão em qualquer momento.

Figura 23 – *Pop-up* de confirmação de exclusão de conta



Fonte: elaborado pelos autores.

Ao realizar o *Logout*, encerra-se a sessão do usuário removendo *token* e *usuario* do `localStorage` e redirecionando de forma segura para a tela de *login*, conforme mostra o Quadro 34.

Quadro 34 – Função de *logout* em `UserPopover.jsx` ou similar

```
import { useNavigate } from 'react-router-dom';

function logout() {
  localStorage.removeItem('token');
  localStorage.removeItem('usuario');
  navigate('/login');
}
```

Fonte: elaborado pelos autores.

3.2.4 Teste de Usabilidade

Para avaliar as funcionalidades da plataforma WalletAI foi conduzido um teste de usabilidade com participantes de perfis variados, com o objetivo de mensurar a aceitação e a eficácia da interface sob a perspectiva de potenciais usuários. O desenho do experimento contemplou três instrumentos complementares: (a) um questionário de perfil do participante; (b) um roteiro de atividades (lista de tarefas) para execução prática; e (c) um questionário de usabilidade baseado em escala Likert. Essa combinação permitiu uma análise abrangente, integrando características dos usuários, observação direta do comportamento em tarefas reais e medidas subjetivas de satisfação.

- a) perfil do participante: antes da execução das tarefas, cada participante respondeu a um formulário de identificação que coletou dados demográficos e informacionais relevantes para a análise — faixa etária, escolaridade, ocupação, familiaridade com ferramentas digitais e experiência prévia com sistemas de gestão financeira ou

carteiras de investimento. Essas informações permitem estratificar os resultados e correlacionar desempenho e percepções com o nível de experiência tecnológica dos avaliadores.

- b) roteiro de atividades: cada avaliador recebeu um conjunto de tarefas representativas do uso real da plataforma, projetadas para avaliar navegação, descoberta de funcionalidades e completude de fluxos críticos. Entre as tarefas definidas estavam: criar uma conta e efetuar *login*; cadastrar e editar um paciente/cliente; iniciar e submeter uma solicitação (fluxo de otimização/criação de carteira); consultar histórico e aplicar filtros de busca; e encerrar sessão (*logout*). Para cada tarefa foi registrado o sucesso (conclusão), o tempo gasto e eventuais comentários do participante, permitindo identificar pontos de atrito, erros recorrentes e necessidades de melhoria na interação.
- c) questionário de usabilidade: após a execução das tarefas, os participantes responderam um questionário composto por itens em escala Likert (1–5) que avaliavam aspectos como facilidade de navegação, clareza das informações, eficiência das funcionalidades, qualidade do feedback do sistema e satisfação geral. Complementarmente, houve espaço para respostas abertas, onde os avaliadores puderam registrar dificuldades encontradas, sugestões de melhoria e avaliações qualitativas sobre a utilidade da ferramenta no contexto de trabalho.

A primeira etapa do estudo investigou o perfil de 18 participantes. Observou-se uma predominância de jovens: 64,7% dos respondentes situaram-se na faixa etária entre 18 e 25 anos, enquanto 11,8% tinham entre 36 e 45 anos e 17,6% entre 46 e 56 anos; apenas 5,9% tinham menos de 18 anos e nenhum participante declarou ter mais de 56 anos ou pertencer à faixa de 26–35 anos. Quanto ao uso de plataformas digitais financeiras/relacionadas a investimento, a amostra mostrou familiaridade prática com essas ferramentas: 47,1% informaram utilizá-las frequentemente, 23,5% relataram uso ocasional (“às vezes”), 23,5% responderam raramente e 5,9% afirmaram nunca utilizar. Os resultados completos estão apresentados na Tabela 1.

Tabela 1 – Perfil dos usuários

Pergunta	Respostas	Quantidade
Faixa etária	Menos de 18 anos	5.9%
	Entre 18 e 25 anos	64.7%
	Entre 26 e 35 anos	0%
	Entre 36 e 45 anos	11.8%
	Entre 46 e 56 anos	17.6%
	Mais de 56 anos	0%
Frequência que utiliza aplicativos/portais financeiros ou de investimento	Nunca	5.9%
	Raramente	23.5%
	Às vezes	23.5%
	Frequentemente	47.1%

Fonte: elaborado pelos autores.

A segunda etapa do questionário consistiu em um roteiro passo-a-passo para conduzir os participantes pelas principais funcionalidades da plataforma WalletAI, contemplando ações que um agente (usuário) da aplicação realiza na prática: cadastrar/alterar dados do usuário, criar uma solicitação (fluxo de otimização/criação de carteira), operar ações que refletem em sua conta (perfil/*logout*). Para cada tarefa registrou-se se o participante concluiu com sucesso. Os resultados agregados encontram-se na Tabela 2.

Tabela 2 – Respostas da execução da lista de tarefas

Tarefas/Respostas	Sim	Não
Criar uma conta (se aplicável) e efetuar <i>login</i>	100%	-
Navegar pelo dashboard	88.9%	11.1%
Iniciar a criação de uma carteira, preencher parâmetros e submeter.	100%	-
Abrir detalhe de uma carteira existente e interpretar composição.	100%	-
Alterar dados do perfil e (se desejado) realizar exclusão de conta.	88.9%	11.1%
Encerrar sessão/deslogar com segurança.	100%	-

Fonte: elaborado pelos autores.

A partir da Tabela 2 observa-se que a maioria dos participantes executou com sucesso as tarefas solicitadas. No entanto, alguns usuários relataram dificuldades no momento de executar a exclusão da conta, especificamente ao encontrar a maneira de manipular o perfil logado. Este problema ocorre pela falta de informações e clareza no web site. Além disso, outros usuários relataram dificuldade na clareza ao navegar pelo dashboard, alegando falta de informações em como prosseguir com a utilização do sistema.

Por fim, na etapa destinada à avaliação das funcionalidades da plataforma, os participantes responderam a um conjunto de afirmações utilizando uma escala Likert de cinco pontos, variando de “discordo totalmente” a “concordo totalmente”. O objetivo dessa fase foi mensurar a percepção dos usuários quanto à atratividade visual da interface, clareza dos elementos, facilidade de navegação, eficiência das ações executadas, qualidade do *feedback* fornecido pelo sistema e satisfação geral com o uso da plataforma. A Tabela 3 apresenta os resultados obtidos nessa avaliação.

Tabela 3 – Respostas referentes as funcionalidades da aplicação

Perguntas / Critérios de avaliação	Discordo totalmente	Discordo parcialmente	Não concordo nem discordo	Concordo parcialmente	Concordo totalmente
O design da interface é visualmente atraente.	-	-	5.6%	22.2%	72.2%
É fácil navegar entre as telas principais (login, dashboard, carteiras, perfil).	-	-	-	22.2%	77.8%
Os ícones e rótulos são claros e intuitivos.	-	-	5.6%	16.7%	77.8%
O processo de cadastro é simples e rápido.	-	-	-	5.6%	94.4%
É fácil encontrar e interpretar as informações sobre a composição das carteiras.	-	-	-	22.2%	77.8%
As ações (criar, editar, excluir) são fáceis de localizar e executar.	-	-	-	27.8%	72.2%
O sistema responde em tempo aceitável às minhas ações (sem lentidão excessiva).	-	-	5.6%	16.7%	77.8%
As mensagens de erro e ajuda (feedback) são claras e úteis.	-	-	5.6%	22.2%	72.2%
Eu me senti seguro(a) ao utilizar a plataforma (privacidade e controle).	-	-	-	5.6%	94.4%
Eu recomendaria esta plataforma para outras pessoas interessadas em gerenciar carteiras.	-	-	-	-	100%

Fonte: elaborado pelos autores.

Com base nos dados apresentados, observa-se que: (i) a interface foi amplamente considerada visualmente atraente, com 72,2% dos participantes concordando totalmente com essa afirmação; (ii) a navegação entre as telas principais também foi percebida como simples e intuitiva, concentrando 77,8% de concordância total; (iii) os ícones e rótulos utilizados foram avaliados de forma bastante positiva, somando 77,8% de concordância total; (iv) o processo de cadastro destacou-se como uma das funcionalidades mais bem avaliadas, visto que 94,4% dos usuários concordaram totalmente que ele é simples e rápido.

Além disso, (v) a maioria dos participantes relatou facilidade para encontrar e interpretar informações sobre a composição das carteiras, refletida em 77,8% de concordância total; (vi) ações como criar, editar e excluir carteiras também foram consideradas fáceis de localizar e executar por 72,2% dos respondentes; (vii) o desempenho do sistema foi percebido como satisfatório, uma vez que 77,8% concordaram totalmente que o tempo de resposta foi adequado.

No que diz respeito ao suporte durante o uso, (viii) as mensagens de erro e feedback foram avaliadas como claras e úteis por 72,2% dos usuários; (ix) a sensação de segurança e privacidade ao utilizar a plataforma também apresentou um índice elevado de aprovação, alcançando 94,4% de concordância total. Finalmente, (x) todos os respondentes afirmaram que recomendariam a plataforma a outras pessoas interessadas em gerenciar carteiras, registrando 100% de concordância total.

Esses resultados demonstram que a plataforma apresenta um alto nível de aceitação e eficácia do ponto de vista funcional. Os dados reforçam que as principais interações (navegação, compreensão das telas, clareza das informações e execução das ações) são percebidas como satisfatórias e coerentes com as necessidades dos usuários. Ao mesmo tempo, a unanimidade em relação à recomendação da plataforma evidencia sua utilidade prática e seu potencial para apoiar de forma eficaz o gerenciamento de carteiras.

4 CONCLUSÕES

A construção de carteiras de investimento que conciliem retorno, risco e restrições práticas continua sendo um desafio relevante para investidores e assessores, sobretudo quando se busca personalização por perfil e controle explícito de riscos extremos. Neste contexto, o presente trabalho abordou a necessidade de métodos capazes de gerar recomendações reproduzíveis, verificáveis e alinhadas às preferências do decisor, integrando desde a extração automatizada de dados até a entrega de carteiras por meio de uma interface acessível.

O objetivo geral, desenvolver uma plataforma de otimização personalizada de carteiras baseada em Algoritmo Genético Multiobjetivo, foi plenamente atendido. Os objetivos específicos também foram contemplados: (i) a extração e preparação dos dados históricos foram automatizadas por meio da biblioteca `yfinance` e de rotinas de limpeza e cálculos de retornos; (ii) as estatísticas centrais (vetor de retornos médios e matriz de covariância) foram calculadas e utilizadas como insumo do problema de otimização; (iii) implementou-se o núcleo de otimização com R-NSGA-II, por meio da biblioteca `pymoo`, incluindo amostragem inicial por Dirichlet, operadores de *crossover* e *mutation* adaptados ao *simplex* e mecanismos para impor cardinalidade; (iv) realizou-se o *tuning* de hiperparâmetros com suporte do indicador R-Hypervolume; (v) procedeu-se à avaliação por *backtests* e comparações com *benchmarks*; e (vi) desenvolveu-se uma API REST (Flask) e uma interface web (React) para disponibilizar o serviço ao usuário final. Cada objetivo foi atendido de forma articulada, resultando em um protótipo funcional e passível de replicação.

Do ponto de vista metodológico, as técnicas adotadas mostraram-se adequadas ao escopo do trabalho. A escolha do R-NSGA-II foi justificada pela necessidade de guiar a busca para regiões de interesse definidas pelo decisor, superando a limitação do NSGA-II clássico de apenas buscar cobertura uniforme da fronteira. A geração inicial por Dirichlet garantiu (i) vetores viáveis no *simplex* sem necessidade de correções posteriores; (ii) operadores adaptados preservaram soma unitária e cardinalidade; (iii) o uso do CVaR como objetivo de risco capturou perdas extremas de forma coerente com práticas modernas de gestão; e (iv) o R-Hypervolume ofereceu critério sólido para comparar configurações. Em suma, a combinação entre algoritmos evolutivos e métricas de risco mostrou-se pertinente para o problema proposto.

Os resultados empíricos indicam que a plataforma alcançou seus propósitos práticos. As carteiras produzidas atenderam coerentemente aos perfis conservador, moderado e arrojado, exibindo o *trade-off* esperado entre retorno e risco (variância e CVaR). As comparações de *backtest* com *benchmarks* de mercado evidenciaram desempenho competitivo das carteiras

geradas em diversos cenários, enquanto a seleção por pontos de referência proporcionou soluções mais alinhadas às aspirações do usuário. Os testes de usabilidade da aplicação web apontaram boa aceitação, confirmando que a solução integra tanto aspectos algorítmicos quanto de usabilidade, sendo capaz de apoiar investidores de diferentes níveis de experiência na tomada de decisões mais informadas e personalizadas.

Ao comparar com trabalhos correlatos, destaca-se o EvoFolio (Guarino *et al.*, 2024), que também emprega algoritmos evolutivos multiobjetivo para construir carteiras personalizadas com base nas preferências do investidor. Assim como neste trabalho, o EvoFolio integra múltiplos critérios no processo de otimização e busca gerar portfólios equilibrados entre risco e retorno. Contudo, enquanto Guarino *et al.* (2024) utilizam o NSGA-II padrão e dependem de o investidor indicar ativos favoritos (*views*), acionando um mecanismo de elitismo que preserva e prioriza a reprodução de carteiras concentradas nesses papéis, delegando a eficácia da otimização à qualidade da *expertise* humana, a presente solução remove essa subjetividade ao utilizar o R-NSGA-II. Neste modelo, a personalização é definida matematicamente por pontos de referência associados a perfis de risco (conservador, moderado ou arrojado), não precisando ter um conhecimento prévio dos ativos e garantindo que a otimização ocorra estritamente dentro da zona de tolerância do investidor. Além disso, o EvoFolio limita-se a controlar a volatilidade média, o que pode ser insuficiente em crises; em contrapartida, este trabalho inova ao integrar o CVaR como terceiro objetivo, oferecendo uma camada superior de segurança que mitiga explicitamente o risco de cauda e protege o patrimônio contra eventos extremos de mercado.

Entretanto, algumas limitações devem ser ressaltadas. O uso de séries históricas está sujeito à dependência temporal e a regimes de mercado não estacionários, o que afeta (i) a robustez das estimativas de retorno e covariância; (ii) a estimação empírica do CVaR pode ser sensível a amostras pequenas e eventos extremos; (iii) custos de transação, liquidez e impactos de mercado não foram incorporados nas simulações, o que reduz a fidelidade do *backtest* a operações reais; (iii) e a restrição de cardinalidade, embora operacionalmente necessária, introduz complexidade computacional que pode limitar escalabilidade em universos muito amplos de ativos. Além disso, a eficiência computacional e o tempo de execução do AGMO em configurações maiores demandam consideração prática antes de um uso em produção em larga escala.

Por fim, como trabalhos futuros, sugere-se: (i) implementar rebalanceamento automático e um módulo para registrar compras/vendas do usuário, de modo a manter a carteira real sincronizada com a recomendação otimizada; (ii) integrar modelos preditivos baseados em redes neurais para melhorar as projeções de retorno e volatilidade e tornar a otimização sensível

a cenários previstos; (iii) incorporar custos de transação, restrições de liquidez e modelagem de impacto para aumentar a aplicabilidade prática; (iv) explorar abordagens híbridas que combinem previsões de séries temporais com otimização robusta; e (v) ampliar experimentos com janelas móveis e validação fora da amostra para avaliar a estabilidade temporal das recomendações. Essas evoluções têm o potencial de transformar a plataforma em um assistente inteligente de investimento, mais dinâmico e alinhado ao comportamento real dos mercados e dos investidores.

REFERÊNCIAS

- ASSAF NETO, Alexandre. **Mercado Financeiro**. 2008, p. 228.
- BODIE, Zvi; KANE, Alex; MARCUS, Alan. J. **Investments**. 9. ed. New York: McGraw-Hill Education, 2014.
- COELLO, Carlos. A. Coello; LAMONT, Gary. B.; VAN VELDHUIZEN, David. A. **Evolutionary Algorithms for Solving Multi-Objective Problems**. Springer, 2007.
- CORNUEJOLS, Gérard; TUTUNCU, R. **Optimization methods in finance**. Cambridge: Cambridge University Press, 2007.
- DARWIN, Charles. **On the Origin of Species**. John Murray, 1859.
- DEB, Kalyanmoy *et al.* **A fast and elitist multiobjective genetic algorithm: NSGA-II**. **IEEE Transactions on Evolutionary Computation**, v. 6, n. 2, p. 182–197, 2002.
- DEB, K.; JAYAVELMURUGAN, S. **Reference point based multi-objective optimization using**, v. 2, p. 635–642, 2006.
- DEB, Kalyanmoy; SUNDAR, Jayant. **Reference point based multi-objective optimization using evolutionary algorithms**. *International Journal of Computational Intelligence Research*, v. 2, n. 3, p. 273–286, 2006.
- ELTON, Edwin J. *et al.*, **Modern portfolio theory and investment analysis**. 9. ed. New York: Wiley, 2014.
- FABOZZI, Frank J.; FOCARDI, Sergio. M.; KOLM, Petter. N. **Financial modeling of the equity market: from CAPM to cointegration**. Hoboken: John Wiley & Sons, 2007.
- FILATOVAS, Ernestas; KURASOVA, Olga; SINDHYA, Karthik. **Synchronous R-NSGA-II: an extended preference-based evolutionary algorithm for multi-objective optimization**. *Informatica, Vilnius*, v. 26, n. 1, p. 33–50, 2015.
- GARCEZ, Matheus. **Aplicações da Inteligência Artificial na Análise e Recomendação de Investimentos**. 2022. Trabalho de Conclusão de Curso (Bacharelado em Matemática) – Universidade Federal de Santa Catarina, Florianópolis, 2022. Disponível em: https://repositorio.ufsc.br/bitstream/handle/123456789/61457/TCC_Garcez_Mat10106251.pdf. Acesso em: 16 abr. 2025.
- GOLDBERG, David. E. **Genetic Algorithms in Search, Optimization and Machine Learning**. Addison-Wesley, 1989.
- GUARINO, Alessandro; PARDO, Sebastián; DELGADO, Marcelo. **EvoFolio: a portfolio optimization method based on multi-objective evolutionary algorithms**. *Neural Computing and Applications*, v. 36, p. 1–26, 2024.
- MARKOWITZ, Harry. **Portfolio selection**. *The Journal of Finance*, v. 7, n. 1, p. 77–91, 1952.
- MICHALIEWICZ, Z. **Genetic Algorithms + Data Structures = Evolution Programs**. Springer, 1996.
- OLIVEIRA, Felipe. S. **A moderna teoria de carteiras: limitações e extensões**. *Revista de Administração de Empresas*, v. 50, n. 2, p. 220–231, 2010.
- PARETO, Vilfredo. **Manuale di Economia Politica**. Società Editrice, 1906.
- PAVÃO, Emerson A. F. **Inteligência artificial aplicada ao mercado financeiro: conceitos, aplicações e desafios**. São Paulo: Aya Editora, 2023. Disponível em: <https://ayaeditora.com.br/livros/L597.pdf>. Acesso em: 16 abr. 2025.

PÚBLIO, Caio C. R. **A inteligência artificial aplicada ao mercado de capitais: o uso de algoritmos no processo decisório de investimentos**. 2022. Trabalho de Conclusão de Curso (Bacharelado em Ciências Econômicas) – Pontifícia Universidade Católica de Goiás, Goiânia, 2022. Disponível em:

<https://repositorio.pucgoias.edu.br/jspui/bitstream/123456789/7894/1/TCC%20II%20P%C3%BAblio%20corrigido%2024.06.pdf>. Acesso em: 16 abr. 2025.

SANTOS, Rafael. **Análise crítica da teoria moderna do portfólio: pressupostos, limitações e alternativas**. Revista de Economia e Administração, v. 9, n. 1, p. 50–63, 2010.

VIEIRA, Ana C. **O uso de algoritmos genéticos na alocação de portfólios de investimentos**. 2021. Disponível em:

<https://repositorio.ufms.br/retrieve/311bc58c-63ba-455d-b907-f4cc0d5d8ed0/10545.pdf>. Acesso em: 16 abr. 2025.

ZITZLER, Eckart; THIELE, Lothar. **Evolutionary Algorithms for Multiobjective Optimization: Methods and Applications**, v. 3, n. 4, p. 257–271, 1999.